LUK: Empowering Log Understanding With Expert Knowledge From Large Language Models

Lipeng Ma, Weidong Yang, *Member, IEEE*, Sihang Jiang, Ben Fei, *Member, IEEE*, Mingjie Zhou, Shuhao Li, Mingyu Zhao, Bo Xu, and Yanghua Xiao

***Abstract*—Logs play a critical role in providing essential information for system monitoring and troubleshooting. Recently, with the success of pre-trained language models (PLMs) and large language models (LLMs) in natural language processing (NLP), smaller PLMs (such as BERT) and LLMs (like GPT-4) have become the current mainstream approaches for log analysis. Despite the remarkable capabilities of LLMs, their higher cost and inefficient inference present significant challenges in leveraging the full potential of LLMs to analyze logs. In contrast, smaller PLMs can be fine-tuned for specific tasks even with limited computational resources, making them more practical. However, these smaller PLMs face challenges in understanding logs comprehensively due to their limited expert knowledge. To address the lack of expert knowledge and enhance log understanding for smaller PLMs, this paper introduces a novel and practical knowledge enhancement framework, called LUK, which acquires expert knowledge from LLMs automatically and then enhances the smaller PLM for log analysis with the expert knowledge. LUK can take full advantage of both types of models. Specifically, we design a multi-expert collaboration framework based on LLMs with different roles to acquire expert knowledge. In addition, we propose two novel pre-training tasks to enhance the log pre-training with expert knowledge. LUK achieves state-of-the-art results on different log analysis tasks, and extensive experiments demonstrate that expert knowledge from LLMs can be utilized more effectively to understand logs. Our source code and detailed experimental data are available at https://github.com/LeaperOvO/LUK.**

***Index Terms*—Log understanding, large language model, pre-trained model, knowledge enhancement.**

Received 14 May 2025; revised 21 July 2025; accepted 27 July 2025. Date of publication 31 July 2025; date of current version 17 October 2025. This work was supported in part by the Natural Science Foundation of Shanghai under Grant 24ZR1405000. Recommended for acceptance by Y. Wang. *(Corresponding authors: Weidong Yang; Bo Xu.)*

Lipeng Ma, Weidong Yang, Sihang Jiang, Mingjie Zhou, Shuhao Li, and Yanghua Xiao are with the School of Computer Science, Fudan University, Shanghai 200433, China (e-mail: lpma21@m.fudan.edu.cn; wdyang@fudan.edu.cn; jiangsihang@fudan.edu.cn; mjzhou19@fudan.edu.cn; shli23@m.fudan.edu.cn; shawyh@fudan.edu.cn).

Ben Fei is with the Department of Information Engineering, The Chinese University of Hong Kong, Hong Kong 999077 (e-mail: benfei@cuhk.edu.hk).

Mingyu Zhao is with the School of Artificial Intelligence, Optics and Electronics (iOPEN), Northwestern Polytechnical University, Xi'an 710072, China (e-mail: myzhao@nwpu.edu.cn).

Bo Xu is with the School of Computer Science and Technology, Donghua University, Shanghai 201620, China (e-mail: xubo@dhu.edu.cn).

Digital Object Identifier 10.1109/TSE.2025.3594046

I. Introduction

With the increasing complexity and scale of IT systems, the maintenance and operation of large-scale systems become more challenging. Logs record the valuable runtime status, they play a crucial role in troubleshooting and enable engineers to monitor system health effectively. However, the increasing volume of logs has made manual analysis a harder task [1]. Consequently, numerous automated log analysis methods utilizing machine learning (ML) or deep learning (DL) models have been proposed to automatically analyze logs, encompassing various tasks such as log parsing [2], [3], [4], [5], anomaly detection [6], [7], [8], [9], [10], [11], [12], root cause analysis [13], [14], [15], failure prediction [16], [17], [18], etc. In particular, with the recent success of pre-trained language models (PLMs) in natural language processing (NLP), especially the advent of large language models (LLMs) represented by ChatGPT and GPT-4 [19], language models (LMs) have garnered significant attention in log understanding and these LM-based approaches [20], [21], [22], [23], [24], [25], [26], [27], [28], [29], [30] achieve tremendous achievements in automated log analysis due to their outstanding performance.

As language models increase in scale and gain enhanced capabilities, two mainstream paradigms have emerged for utilizing LMs in log understanding. The first one is based on PLMs, which follows the *pre-train & fine-tune* paradigm, such as BERT [31], fine-tuning a PLM with task-specific data and enabling it to specialize in the given task. Considering the need for fine-tuning with limited computational resources and cost savings, LMs in this paradigm are typically smaller in size, such as BERT with 110M parameters. The second one is based on LLMs with vast scale and complexity[1], which can tackle a diverse array of tasks with remarkable proficiency. The LLM-based methods follow the *In-Context Learning* (ICL) paradigm [32], [33], learning from a few examples in the context without updating the parameters.

Despite the powerful performance of LLMs, limited accessibility and higher costs present significant challenges in leveraging the full potential of LLMs. These LLMs often come with substantial usage fees and have restricted access to data privacy and security. Even if the LLM is deployed locally, it still may face resource constraints, making LLMs less attainable

[1]We use LLM to specifically refer to the large language models over 10B parameters that utilize the ICL paradigm without fine-tuning the parameters.

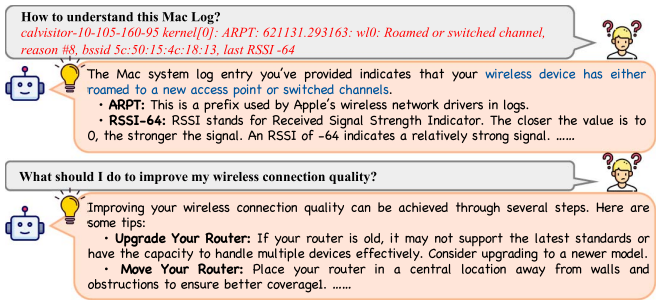


Fig. 1. Example of prompting ChatGPT to help understand log, which can provide detailed background information, terminology explanations, and solutions. Without knowledge, the beginner fails to understand the RSSI value.

for individuals and smaller organizations. In addition, due to their complex autoregressive architectures and a large number of parameters, LLM-based methods are computationally inefficient. Considering that modern software systems can produce several petabytes per day [34], [35], directly employing LLMs for log analysis is impractical due to the significant overhead of querying LLMs, such as inference time and network latency [29], [36]. In contrast, smaller PLMs with smaller parameters, requiring limited resource costs, are more efficient than LLMs [37]. Hence, we argue that the smaller PLM is more practical in extensive log analysis scenarios [23], [24], [25].

However, the smaller PLM encounters a bottleneck in log understanding due to the inadequacy of expert knowledge. Logs inherently employ concise and highly specialized terminology, the smaller PLM struggles to understand valuable information from logs like experts [22]. This issue can be attributed to the lack of rich sources for knowledge acquisition. Although KnowLog proposes to acquire expert knowledge from documentation [22], most logs lack professional documentation (e.g., almost unable to find relevant documents for logs coming from Loghub [38]), and the manual construction of knowledge is costly and inefficient, which significantly hinders the availability of the model.

Fortunately, LLMs with extensive knowledge can understand logs comprehensively, attributed to training on massive textual data related to code [39] and logging [40], [41]. For example, as depicted in Fig. 1, ChatGPT can better assist in understanding the Mac log. Consequently, LLMs such as GPT-4 compensate for the lack of knowledge of the smaller PLM, which can analyze logs more efficiently and professionally.

To more effectively harness the knowledge embedded within LLMs for log understanding, we argue that expert knowledge can be elicited from LLMs and subsequently utilized to empower the smaller PLM, making the smaller model understand logs like experts. However, two main challenges exist in enhancing log understanding with expert knowledge from LLMs. (1) **Effective Knowledge Acquisition:** Although LLMs have shown remarkable capabilities in various tasks, LLMs still have difficulty in acquiring complete and accurate knowledge due to hallucination, which may result in not covering all the depth of knowledge, and there may be misunderstandings when dealing with domain-specific data [42], [43]. In addition, in the era of LLMs, knowledge distillation (KD) [44] emerges as a pivotal methodology for transferring advanced capabilities from LLMs to smaller models. Compared to obtaining answers directly from the teacher's model, distillation towards the reasoning process has proved to be a more effective method [44], [45]. However, current knowledge distillation methods, such as COT-based KD [46], [47], struggle to acquire expert knowledge, instead imitating the teacher's reasoning forms and ignoring the premise that reasoning requires expert knowledge. These distillation methods, which emphasize the process of reasoning and outcome, still hardly compensate for the lack of expert knowledge. (2) **Knowledge Enhancement:** Since logs and expert knowledge obtained from LLM are heterogeneous data, smaller models with fewer parameters and limited learning capacity struggle to capture knowledge [48], [49]. Masked Language Modeling (MLM) [31] is the widely used pre-training task for most pre-trained models, where they can only perceive their own contextual information and cannot incorporate external knowledge. In addition, KnowLog proposes an abbreviation prediction task specialized for log understanding. However, its ability to enhance the knowledge of abbreviations in the logs is limited. It cannot adaptively perceive other important information from the knowledge to understand logs, such as the parameter knowledge of logs.

To overcome the above-mentioned challenges, we propose a novel knowledge enhancement framework to empower log understanding on a smaller PLM, called **LUK**. Rather than utilizing LLMs to solve specific tasks directly, LUK first acquires expert knowledge from LLMs, then enhances the log pre-training with the corresponding expert knowledge, and finally, the knowledge-enhanced PLM for logs can be fine-tuned to solve downstream log analysis tasks efficiently. This framework can maximize the advantages of both LLM and smaller PLMs.

Specifically, to solve the first issue, we design a multi-expert collaboration framework to acquire expert knowledge from LLMs. Motivated by the waterfall model [50] in software engineering, we build a professional team consisting of *Director*, *Executor*, and *Evaluator*, and define the identity and responsibility of the roles through prompt engineering, enabling LLMs to think and handle tasks just like role play. Then the team cooperates and interacts to construct expert knowledge. To solve the second issue, we develop a novel hierarchical knowledge-enhanced pre-training framework that combines two complementary training objectives: 1) the word-level token prediction task, focusing on granular knowledge perception, and 2) the sentence-level semantic alignment, enhancing contextual understanding.

To evaluate the effectiveness of LUK, we conduct experiments on the software system and network device logs, including six log analysis downstream tasks. The experiment results show that LUK can harness knowledge from LLMs more effectively to improve log understanding and achieve state-of-the-art results on different log analysis tasks. In addition, LUK exhibits remarkable generalization and robustness in log analysis, with notable advantages in low-resource scenarios.

This work is an extension of our conference paper (KnowLog) published in ICSE 2024 [22]. We make several new contributions:

1) LLM-Driven Knowledge Acquisition: This work designs a multi-expert collaboration framework to distill expert knowledge from LLMs, which compensates for the expensive overhead of LLMs. In addition, this collaboration framework mitigates issues of hallucination and incomplete knowledge by simulating the roles of Director, Executor, and Evaluator, enabling LLMs to construct reliable and comprehensive domain-specific knowledge. Compared to the limited usage scenario of KnowLog, which relies on documentation created by human experts to acquire knowledge, this work focuses on more generalized log analysis scenarios where logs are available without accessible domain knowledge.
2) Hierarchical Knowledge Enhancement: This work proposes a hierarchical knowledge-enhanced pre-training framework, which incorporates two complementary training objectives: (1) word-level token prediction for granular knowledge perception and (2) sentence-level semantic alignment for improved contextual understanding. This dual-objective approach enables smaller PLMs to more effectively utilize expert knowledge from heterogeneous data, addressing their inherent limitations. Compared to KnowLog's inability to perceive fine-grained knowledge beyond abbreviation, this work enables a more comprehensive understanding of fine-grained knowledge from the provided information.
3) Extensive Evaluation: This work achieves state-of-the-art results on different log understanding tasks, proving the effectiveness of acquiring knowledge from LLMs to enhance log understanding. Moreover, LUK extends its applicability by focusing on challenging scenarios such as unstable log data and low-resource settings. Experiments show that LUK consistently achieves superior performance across these scenarios, demonstrating its robustness and adaptability.

II. Related Work and Motivation

A. Pre-Trained Language Models for Log Analysis

Pre-training is the key to the success of the PLMs and LLMs, where a language model is first trained on an extensive corpus with effective pre-training tasks to capture general knowledge and then adapted to solve different downstream tasks [51], [52]. The emergence of BERT [31] heralds the success of the *pre-train & fine-tune* paradigm in natural language. Such language models improve the model's ability on a specific task by fine-tuning using task-specific objective functions after pre-training. Many studies [53], [54], [55] have demonstrated that pre-training on domain corpus and fine-tuning with supervised data can improve performance on a specific task, even outperforming LLMs [56], [57]. In the field of log analysis, many studies, including pre-training on log corpus [20], [21], [22], [58] and fine-tuning on log analysis tasks [24], [25] have demonstrated the effectiveness of the pre-trained model for log analysis.

However, training solely on a general or log corpus struggles to further improve log understanding due to the lack of expert knowledge. KnowLog [22] firstly proposes to enhance log understanding by integrating expert knowledge during the pre-training phase. Nevertheless, KnowLog's usage scenarios are limited as it heavily relies on documentation created by human experts to acquire knowledge. In contrast, this work addresses more generalized log analysis scenarios where logs are available without accompanying domain knowledge. In addition, these pre-trained models rely on the existing Masked Language Modeling (MLM) as the pre-training task, where they only perceive their own contextual information and are unable to integrate external knowledge. Although KnowLog also proposes the abbreviation prediction task, this method cannot adaptively perceive other knowledge than the abbreviation. Considering the abundance of knowledge required for log understanding, hence, we also investigate making the PLM adaptive in perceiving useful information from the knowledge during pre-training.

B. Large Language Model for Log Analysis

Large language models (LLMs), representing a remarkable advancement in artificial intelligence, have been trained on diverse corpora, enabling them to generate human-like text and provide accurate responses to queries [59]. Notably, the recent introduction of transformative models like ChatGPT and GPT-4, characterized by their enormous scale and alignment with human feedback, presents a novel opportunity for enhancing software engineering tasks [23], [60], [61], [62]. LLMs can learn from the prompt context without training the model, which is called In-context Learning (ICL) [63]. Furthermore, research has shown that augmenting the size of LLMs significantly boosts their capacity for knowledge encoding and reasoning [48], [64]. Due to the outstanding abilities of LLMs, many recent studies [26], [27], [28], [29], [30] utilize the ICL paradigm of LLMs for different log analysis tasks without fine-tuning the model and make tremendous achievements in log analysis tasks.

However, restricted accessibility, higher costs, and computational inefficiency limit the widespread use of LLM in real-world scenarios. For example, the GPT-3 model with 175 billion parameters requires 326GB of GPU memory to deploy [65]. Given the enormous data volumes produced by modern software systems daily, the overhead of querying LLMs, including inference time and network latency, renders direct application of LLMs for log analysis impractical. In contrast, small PLMs with smaller parameters are more efficient compared to LLMs [37]. We argue that the smaller PLM can make predictions better in log analysis tasks given sufficient expert knowledge. In this paper, we guide LLMs as domain experts with prompts and explore how to effectively acquire and utilize expert knowledge from LLMs to empower log understanding.

III. Method

A. Overview

Fig. 2 shows the conceptual overview of LUK, which consists of three phases: knowledge acquisition, knowledge-enhanced

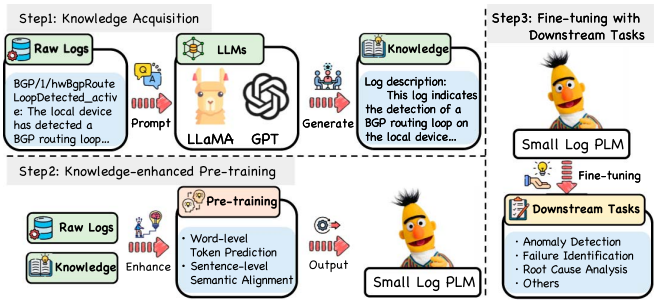


Fig. 2. Conceptual overview of LUK.

pre-training, and fine-tuning with downstream tasks. Specifically, in the first stage, we collect various logs as input, and then we design a multi-expert collaboration framework based on LLMs to acquire expert knowledge of logs. In the second stage, we take the raw logs and the corresponding expert knowledge as input. To effectively leverage expert knowledge to empower log understanding on a smaller model, we enhance the log pre-training with knowledge based on BERT-base and propose two pre-training tasks on the word level and sentence level. Finally, a specific PLM for logs is obtained, which can be fine-tuned on downstream tasks to solve log analysis tasks. In the following sections, we describe the details of LUK.

B. Multi-Expert Collaboration Framework

Hallucination is an inherent flaw of LLMs, which may lead to incomplete or incorrect generation, ultimately failing to fulfill the intended requirements. To construct comprehensive and accurate expert knowledge with LLM, motivated by *cognitive synergy* [66] and *teamwork theory* [67], humans can leverage the power of collaboration and interaction to solve complex problems. As shown in Fig. 3, we design a multi-expert collaboration (MEC) framework, where we assign LLMs to different roles, and these roles collaborate to generate effective expert knowledge.

1) Role Design: Based on the classical waterfall model [50] in software engineering, we design a similar waterfall model to analyze logs consisting of three stages: analysis, execution, and evaluation. Thus, we build a professional team based on LLMs and define clear goals for each role, comprising a *Director*, *Executor*, and *Evaluator*.

It is widely acknowledged that one key capability of LLMs is to follow input natural language instructions or prompts [68], [69], hence they can customize the output to the specific needs of the user. As shown in Fig. 3, the prompt design methodology involves the pre-definition of role cards to allocate distinct roles and responsibilities to LLMs. These role cards encompass character identity, task objectives, requirements, and input queries. Initially, the LLM is assigned a character identity to emulate a specific expert role. This step enables the model to adjust its output to match the expertise and responsibilities associated with the designated role. Subsequently, the task objective will instruct the LLM to perform the corresponding responsibilities aligned with the assigned character identity. Moreover, the requirement outlines the anticipated outcome or characteristics that the generated content must exhibit. These criteria may encompass aspects such as reasoning strategies, informativeness, or other pertinent factors crucial to the task at hand. Lastly, the inclusion of an input query or data serves to contextualize the task for the LLM by presenting specific information or scenarios that the model should engage with or respond to.

By integrating these components into the prompt design framework, we can effectively guide the LLM in generating responses that are tailored to specific roles, tasks, and desired outcomes. These three different roles are assigned the following tasks:

- ***Director.*** The *Director*'s primary objective is to establish a comprehensive high-level framework aimed at providing a strategic overview to drive the team's efforts in understanding logs. This role outlines the key points essential for log understanding, thus ensuring that essential insights are highlighted and comprehensively addressed.
- ***Executor.*** As the central role of this team, the *Executor* is tasked with the detailed execution of the log analysis process. By receiving key points from the *Director* and feedback from the *Evaluator*, the *Executor* undertakes two critical responsibilities: content generation and refinement. Specifically, the *Executor* first generates detailed content aligned with the outlined key points. Then the *Executor* refines and polishes the content based on feedback, improving the knowledge quality.
- ***Evaluator.*** The *Evaluator* assumes the crucial responsibility of ensuring the quality and adherence of generated content to predefined requirements. We define three evaluation requirements for the *Evaluator* to check the *Executor*'s generation on three fundamental aspects: completeness, consistency, and conciseness. By evaluating content against these criteria, the *Evaluator* validates the accuracy and completeness of the generated knowledge.

2) Dehallucination With Self-Evaluation: Hallucinations seriously affect the accuracy and quality of outputs [70], [71]. Given the tendency of LLM-generated content to exhibit deficiencies in comprehensiveness and informativeness, ensuring the superior quality of generated knowledge poses a significant challenge. To mitigate the hallucination of LLM and ensure the quality of expert knowledge, we build the role of an *Evaluator* in the MEC framework and define three evaluation aspects to evaluate the quality of generated knowledge automatically, which encourages the LLM to actively seek more detailed suggestions from the self-evaluation before delivering a formal response. However, the MEC framework runs under the challenging reference-free setting, and the evaluation requirements cannot be directly quantified through metrics, where the LLM tends to generate ambiguous and inconsistent evaluations without fine-grained distinguishability, causing unsatisfactory evaluation performance [72], [73].

To address the evaluation weaknesses of the LLM, we provide evaluation examples as a reference in the *Evaluator* prompt. By providing specific examples, the *Evaluator* uses them as the basis for evaluation, thus better guiding the

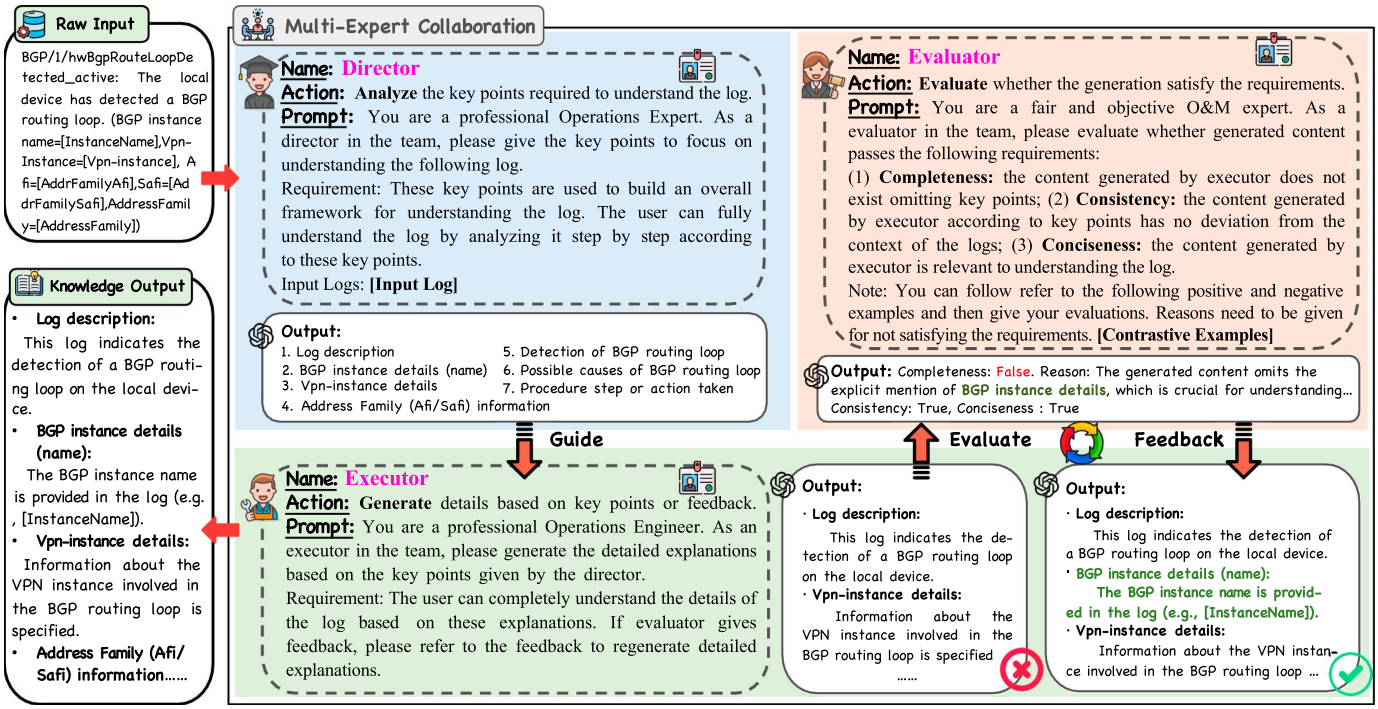


Fig. 3. Framework of multi-expert collaboration, which consists of three experts. First, the *Director* meticulously analyzes the input logs to discern key points. Subsequently, the *Executor* utilizes these key points to create detailed content. Finally, the *Evaluator* evaluates the quality of the generated content. In cases where the evaluation result is unsatisfactory and improvements are needed, the *Evaluator* provides feedback to the *Executor* for refinement.

evaluation of LLMs towards our desired goals. Considering that the single type of example makes the LLM biased, motivated by contrastive learning techniques [74], [75], we construct **contrastive examples** consisting of positive and negative knowledge of the same log to *Evaluator* as reference examples, where the positive example showcases the desired knowledge, while the negative example highlights the unsatisfactory knowledge that should be avoided. By analyzing both types of examples before generating the evaluation, the *Evaluator* can reason about our evaluation requirements and understand how to evaluate them.

Specifically, we select one log L with high-quality knowledge K_{pos} as the positive example, which can be manually constructed by experts or generated from the LLM and manually confirmed. Considering that negative examples $\{K_{neg}\}_{c=1}^{C}, C=3$ of the log L are unavailable directly, to construct the negative examples automatically, we prompt the LLM to modify the positive knowledge, making it unsatisfactory for evaluation. As shown in Fig. 4, we first give the background in the prompt and then clarify the task of modifying knowledge for the LLM, where we can specify which evaluation metrics are not fulfilled. Next, we provide the raw log, the raw positive knowledge, the key points, and the three evaluation requirements in the prompt. Finally, we require the LLM to give the reason for not fulfilling the evaluation requirements while generating the modified content, which not only improves the reasoning ability of the *Evaluator* but also provides guidance for the revision of the content by *Executor*. In cases where the LLM-generated negative examples do not capture the characteristics we want to avoid, this can be manually checked, and feedback can be given to the LLM to re-generate negative samples. To avoid the higher expense of the longer prompt, we construct one positive knowledge and its corresponding three negative knowledge as reference examples to be provided to the *Evaluator*. The contrastive examples we constructed in our experiments are shown in Fig. 6.

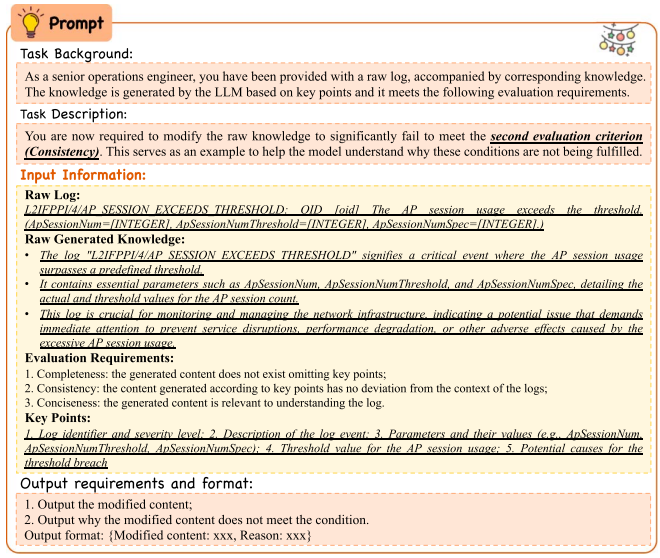


Fig. 4. Prompt template of constructing negative samples, where the underlined portion of the prompt supports user modification based on their own data and intent.

3) Collaboration: After constructing evaluation examples and assigning roles to LLMs, different roles start working to collaborate according to the provided requirements. First, the *Director* carefully analyzes the input logs to recognize key points and formulates a plan for comprehending the logs effectively. Subsequently, the *Director* shares these insights with the *Executor* to guide the content generation process. The *Executor* utilizes these key points to create detailed content that aligns with the requirements. Finally, the detailed content produced by the *Executor* undergoes evaluation by the *Evaluator*. If the content meets the evaluation requirement, it is directly output as expert knowledge. In cases where the evaluation result is unsatisfactory and improvements are needed, the *Evaluator* provides feedback to the *Executor* for refinement. To rigorously ensure the quality of the generated knowledge, this evaluation and refinement mechanism will be iterative until the content generated by *Executor* meets the evaluation requirement or the maximum number of iterations is reached. We set up 3 iterations to build expert knowledge in our experiments. The pseudocode of the multi-expert collaboration framework is outlined in Algorithm 1.

This collaborative framework has two pivotal advantages: Firstly, leveraging multiple roles' collective insights and skills enables a multi-perspective analysis of the input log, thereby mitigating the one-sidedness and bias inherent in a single model's response. Secondly, through the feedback loop mechanism, the framework enables iterative refinement of content. This iterative process of evaluation, feedback incorporation, and content enhancement promotes continuous improvement, thereby ensuring that the acquired expert knowledge is comprehensive and of high quality.

Algorithm 1 Pseudocode of multi-expert collaboration.

Require: Input Log log, LLM $\mathcal{M}$, Example Knowledge K_{pos}
Ensure: Output Expert Knowledge k
# Construct Contrastive Examples
for $c = 1$ to C **do**
$K^c_{neg} = \mathcal{M}(Prompt_{Modify}, K_{pos})$
end for
$K_{examples} = K_{pos} \cup \{K_{neg}\}^C_{c=1}$
Initial Roles
$\mathcal{DIRECTOR} = Init\_Role(Prompt_{Dirctor}, \mathcal{M})$
$\mathcal{EXECUTOR} = Init\_Role(Prompt_{Executor}, \mathcal{M})$
$Prompt_{Evaluator} = Prompt_{Evaluator}.join(K_{examples})$
$\mathcal{EVALUATOR} = Init\_Role(Prompt_{Evaluator}, \mathcal{M})$
Collaboration
Init: $epoch = 0, feedback = False$
$keypoint = \mathcal{DIRECTOR}(log)$
$content = \mathcal{EXECUTOR}(log, keypoint)$
repeat
$epoch = epoch + 1$
$feedback = \mathcal{EVALUATOR}(log, keypoint, content)$
if $feedback == False$ **then**
$content = \mathcal{EXECUTOR}(content, feedback)$
end if
until $epoch == Epoch$ or $feedback == True$
$k = content$
return k

C. Pre-Training With Knowledge Enhancement

The overall pre-training framework is shown in Fig. 5, the input of the pre-training framework is a batch of pairs consisting of logs and the corresponding expert knowledge, then these pairs are fed into two encoders to obtain log representations and knowledge representations through two encoders respectively, where two encoders are initialized with the same pre-trained model. Finally, based on the structural characteristics of the log, we propose two pre-training tasks at word-level and sentence-level: (1) token prediction and (2) semantic alignment, to incorporate expert knowledge for improving log understanding.

1) Word-Level Token Prediction: To sufficiently understand domain terminologies in the logs, we propose a Token Prediction pre-training task (TP). Unlike existing word-level tasks of traditional PLMs only utilizing local context to predict tokens, our proposed word-level task requires models to aggregate context and knowledge for predicting tokens, leading to a knowledgeable pre-trained model.

Specifically, given a log l and expert knowledge k generated from the LLM, we get the corresponding input token sequence $ls = \{l_0, l_1, ..., l_n\}, ks = \{k_0, k_1, ..., l_m\}$ after tokenization. The token prediction task randomly masks a certain percentage (15% in our experiments) of the log tokens l_i with a special $[MASK]$ token, and then tries to recover them by perceiving external knowledge. To perceive useful information from the generated knowledge, we design a knowledge perception module (KPM) based on the cross-attention mechanism, which fuses the heterogeneous representations of log and knowledge to bridge the knowledge gap. The process of this module can be described in three steps:

Firstly, log encoder and knowledge encoder encode ls and ks, respectively, and get the corresponding token representations $\boldsymbol{l} = \{\boldsymbol{l}_{[CLS]}, \boldsymbol{l_0}, \boldsymbol{l_1}, ..., \boldsymbol{l_n}\}$ and $\boldsymbol{k} = \{\boldsymbol{l}_{[CLS]}, \boldsymbol{k_0}, \boldsymbol{k_1}, ..., \boldsymbol{k_m}\}$.

Secondly, since not all tokens in knowledge contribute equally to the masked token prediction and to measure the importance of each token in knowledge for the token semantic, KPM calculates the semantic similarity between masked token $\boldsymbol{l_i}$ and knowledge $\boldsymbol{k}$, and each token in knowledge is assigned a weight to represent its importance. Considering that the log representation and the knowledge representation come from two different encoders, we introduce the cross-attention mechanism to compute this weight, where queries come from the masked log token $\boldsymbol{l_i}$, keys and values come from the knowledge representations $\boldsymbol{k}$:

$$Q = W_Q \boldsymbol{l_i}, K = W_K \boldsymbol{k}, V = W_V \boldsymbol{k}, \tag{1}$$

where, l_i from the log encoder and k from the knowledge encoder. Then assign a weight α for each token in the knowledge:

$$\alpha = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right), \boldsymbol{k}' = \alpha V, \tag{2}$$

where W_Q, W_K, and W_V are learnable parameter matrices, d_k is the dimension of representation, α refers to the attention

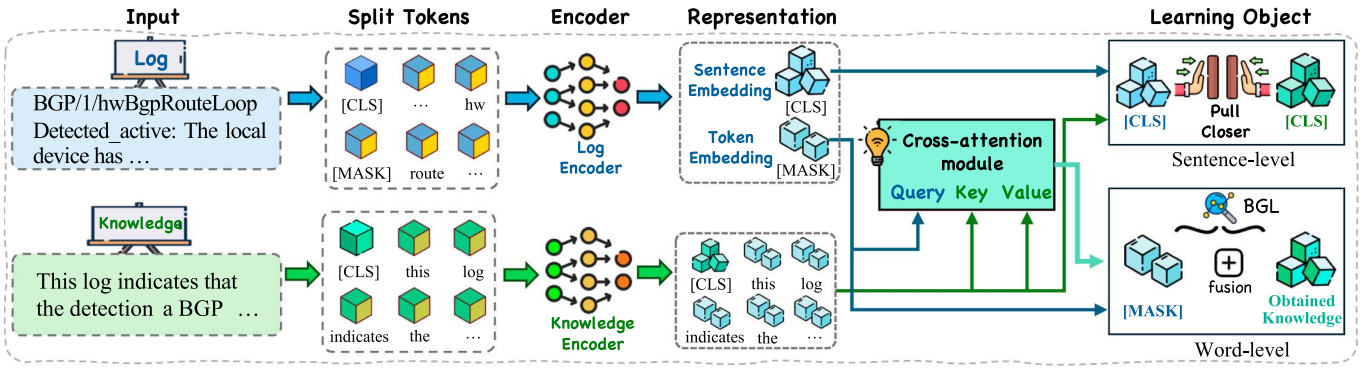


Fig. 5. The overview of LUK pre-training framework. LUK includes a hierarchical knowledge-enhanced framework, including two complementary training objectives on word level and sentence level. The word-level object is designed from the perspective of domain token understanding, while the sentence-level objective is designed from overall semantic alignment.

distribution, and $\boldsymbol{k}^{'}$ denotes the obtained information from the knowledge based on the weights.

Thirdly, masked token $\boldsymbol{l_i}$ and obtained knowledge $\boldsymbol{k}^{'}$ are concatenated inside the d_k-dimensional space to constitute a fused representation $[\boldsymbol{l_i}; \boldsymbol{k}^{'}]$, which is used to predict the original token:

$$\hat{y}_i = softmax(W_f[\boldsymbol{l_i}; \boldsymbol{k}^{'}]), \tag{3}$$

where W_f is the weight parameter. This fused representation contains the key information for the token understanding, which is adaptively acquired from knowledge. At last, the TP objective is to predict the original tokens that are masked out, formulated as follows:

$$\mathcal{L}_{TP}(\theta) = -log\mathbf{p}(l_i|ls\backslash\{l_i\}), \tag{4}$$

where $ls\backslash\{l_i\}$ denotes the log tokens sequence ls with token l_i being masked.

2) Sentence-Level Semantic Alignment: Since logs do not conform to the grammatical structure of human language and are always concise, it is difficult to sufficiently capture the semantics of logs. To enrich the semantic context of logs and improve the robustness of log representations, following KnowLog [22], we construct a Semantic Alignment pre-training task (SA) at the sentence level that aligns the semantic representations of logs with the expert knowledge with contrastive learning.

Specifically, we first construct positive and negative examples from a batch $T = \{(l, k)\}$ of input pairs with size N. (l^a, k^a) is the a-th pair in the batch. We obtain logs l and knowledge k from the same input pairs $\{(l^a, k^b)_{a=b}\}$ as positive examples and different input pairs $\{(l^a, k^b)_{a\neq b}\}$ as negative examples. Then we use $\boldsymbol{l^a}$, $\boldsymbol{k^a}$ to indicate the representations of the log and the knowledge, where the hidden state of the special symbol $[CLS]$ as the sentence representation. To pull closer positive samples and push away negative samples, the training objective of SA is defined as follows:

$$f(\boldsymbol{l^a}, \boldsymbol{k^b}) = exp\left(\frac{(\boldsymbol{l^a})^\top \boldsymbol{k^b}}{||\boldsymbol{l^a}|| \cdot ||\boldsymbol{k^b}||} \Big/ \tau\right), \tag{5}$$

$$\mathcal{L}_{SA}(\theta) = \frac{1}{N}\sum_{a=1}^{N} -log\frac{f(\boldsymbol{l^a}, \boldsymbol{k^a})}{f(\boldsymbol{l^a}, \boldsymbol{k^a}) + \sum_{b=1}^{N-1} f(\boldsymbol{l^a}, \boldsymbol{k^b})}, \tag{6}$$

where τ is the temperature hyperparameter, which is set to 0.05 empirically in our experiments.

At last, we sum the TP loss and the SA loss, and obtain the overall training objective:

$$\min_{\theta} \mathcal{L}_{TP}(\theta) + \mathcal{L}_{SA}(\theta) \tag{7}$$

D. Fine-Tuning With Downstream Tasks

After pre-training, we fine-tune the log encoder on different downstream tasks. We group all downstream tasks into two categories based on the input type: log-single tasks and log-pair tasks. We use the final hidden state of the first token (the $[CLS]$ token) as the sentence representation. For log-single tasks, we input the output of the language model l_{CLS} into a multi-layer perception network function f_H and obtain the prediction as $f_H(l_{CLS})$. For log-pair tasks, we follow sentence-bert [76] and use the language model to encode two separate sentences u, v, then input u_{CLS}, v_{CLS} and element-wise difference $|u_{CLS} - v_{CLS}|$ into a multi-layer perception network function f_H and obtain the prediction as $f_H([u_{CLS}; v_{CLS}; |u_{CLS} - v_{CLS}|])$.

IV. Experiments

A. Data Preparation

In this paper, we collect logs from software systems and network devices to construct expert knowledge and the log pre-trained model. Specifically, we collect software system logs from Loghub [38], including operating system logs composed of Windows and BGL, as well as distributed system logs composed of HDFS and OpenStack. Given that these datasets contain a significant proportion of duplicated logs, we employ widely used Drain [77] to parse these logs and achieve log templates. Subsequently, we only sample one instance log as input that corresponds to each unique template. In addition, we also collect network device logs from the public documentation of two vendors, Cisco[2] and Huawei[3], including three

[2]https://www.cisco.com/c/en/us/support/all-products.html
[3]https://support.huawei.com/enterprise/en/index.html

TABLE I
STATISTICS OF THE DATASET USED FOR KNOWLEDGE GENERATION AND PRE-TRAINING

Datasets	Category	# of Log Templates
Software System	Distributed System	2,292
	Operating System	11,947
Network Device	Cisco	16,591
	Huawei	12,399
Total		43,229

devices: Switches, Routers, and WLAN. The detailed statistics are shown in Table I.

B. Parameters Setting

- As for expert knowledge acquisition, we explore different LLMs for experiments, including two proprietary LLMs: *gpt-3.5-turbo* (ChatGPT) and *gpt-4o* (GPT-4o), and one open-source model: *Llama-3.3-70B-Instruct* (LLama3-70B) [78]. To increase the stability of LLM's output, we set the temperature of LLMs to 0.
- For contrastive examples provided to the *Evaluator*, we show in Fig. 6.
- For pre-training, we use *bert-base-uncased* with 110M parameters in our experiments. During pre-training, we set the batch size as 32, epochs as 40 and the maximum length of the input text as 512. Moreover, the optimizer we adopt is Adam with a learning rate of 5e-5, a weight decay of 0.01, learning rate warmup for 2,000 steps and linear decay of the learning rate after.
- For fine-tuning, we adopt the cross-entropy loss as the loss function in downstream tasks, and we set epoch to 20 and 10 on log-single and log-pair tasks, respectively.
- We spend 3 hours pre-training the LUK on 2 NVIDIA A100 GPUs with PyTorch 2.4.0.

C. Downstream Tasks

To explore the performance of LUK on different log analysis domains, we conduct experiments on different downstream tasks, including software systems and network device logs.

Following prior studies on software system logs [38], [79], we focus on two widely studied tasks, anomaly detection and failure identification. These tasks are representative of traditional log analysis challenges and help validate the core effectiveness of our approach. However, recognizing that existing public datasets for these tasks often exhibit relatively simple and repetitive anomaly patterns [80], [81], we expand our focus to include knowledge-intensive log understanding tasks [22]. These tasks emphasize the understanding of log semantics and require deeper domain knowledge, making them ideal for evaluating the capabilities of knowledge-enhanced models. Following the task construction process on log understanding tasks [22], we construct four different downstream tasks on network device logs across three vendors, and these datasets have been organized and made accessible through our GitHub repository.

The evaluation datasets in our experiments include two types: datasets included in the pre-training corpus and those not included in the pre-training corpus. For most datasets, we collect the evaluation datasets from the same source as the pre-training data. Moreover, apart from logs involved in pre-training, to verify the generalization capability of LUK, we also collect logs that are not included in the pre-training for evaluation. In Tables II and III we provide statistics for different tasks of their datasets and make all publicly available datasets accessible in our GitHub repository. Next, we give an introduction to each task and its evaluation metrics.

1) Downstream Tasks of Software System Logs:

- **Anomaly Detection (AD).** Anomaly detection is a widely researched log analysis task to predict whether anomalies exist within a short period of log messages, where the input is a log sequence and the output is True or False. Following Biglog [21], we concatenate each log in the sequence and then input them to the encoder to obtain the representation of the sequence.
  Dataset and Metric. Following previous anomaly detection studies [12], [82], we collect datasets from Loghub [38]. Given the substantial size of anomaly detection datasets, such as Spirit and Thunderbird, which contain more than 200 million messages, full-scale analysis with LLMs incurs high overhead. Hence, we take 1,000,000 raw logs in chronological order with a window size of 20, then group logs into chronologically ordered datasets (30,000 / 10,000 / 10,000), preserving the natural characteristics of log patterns while preventing data leakage. To measure the effectiveness of different models in anomaly detection, we report *Precision*, *Recall*, and *F1* on the True (anomaly) class as evaluation metrics.
- **Failure Identification (FI).** Failure identification aims to further discern the type of failure present in the anomaly log. Given the log messages, the model is required to determine what error emerges.
  Dataset and Metric. This dataset comes from [18], which is an OpenStack dataset including 396 failure tests and 16 kinds of API errors, such as "network delete error", "openstack network create error". Usually, engineers are interested in whether top-K recommended results contain the correct error, hence, we report the *Recall@K* rate as the evaluation metric.

2) Downstream Tasks of Network Device Logs:

- **Module Classification (MC).** MC is a log-single task aiming at identifying which module the log originates from, which input is a log with the masked module name and the output is the corresponding module name.
  Dataset and Metric. We collect the log from Table III and replace the module name with *[MASK]* as input, the module name in the log as ground truth. Obviously, this is a multi-classification task and the model needs to understand the contextual information of the logs to accurately identify their source. As an unbalanced multi-class classification task and considering the importance of different classes, we report *Accuracy* and *Weighted F1* as evaluation metrics.

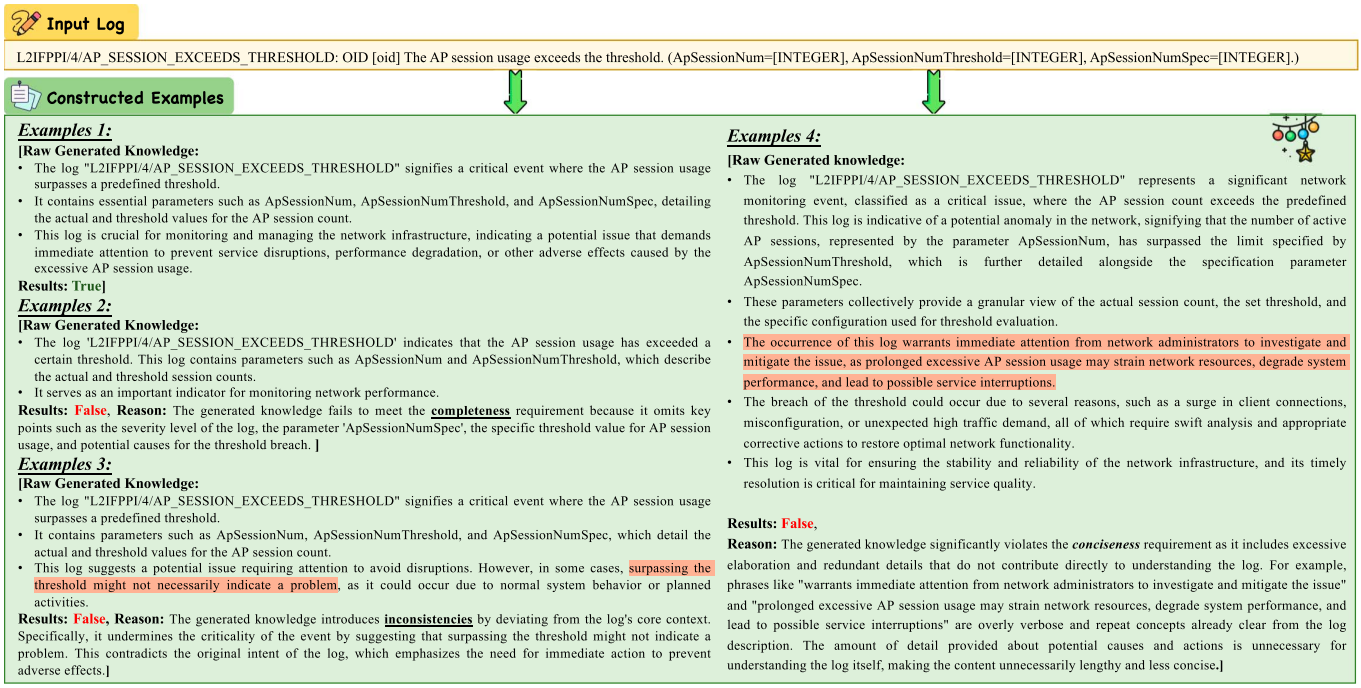


Fig. 6. The contrastive samples constructed in our experiments, where Example 1 represents the positive example, and the remaining three examples are negative examples obtained from the modification of GPT-4o. Colour-marked text indicates that the evaluation requirements are unsatisfied.

TABLE II
STATISTICS OF DOWNSTREAM TASKS DATASETS ON SOFTWARE SYSTEM (TRAINING/VALIDATION/TESTING SIZE)

Tasks	Dataset	#Size
Anomaly Detection	BGL	600,000 / 200,000 / 200,000
	ThunderBird*	600,000 / 200,000 / 200,000
	Spirit*	600,000 / 200,000 / 200,000
Failure Identification	Openstack	236 / 80 / 80

* indicates logs not involved in the pre-training.

TABLE III
STATISTICS OF DOWNSTREAM TASKS DATASETS ON NETWORK DEVICE (TRAINING/VALIDATION/TESTING SIZE)

Tasks		Switches	Routers	Security*
Module Classification	Cisco	13,495/4,498/4,498	7,265/2,422/2,421	-
	Huawei	3,439/1,146/1,146	2,539/846/845	-
	H3C*	1,241/413/413	1,336/445/444	-
Fault Phenomenon Identification	Huawei	362/120/120	-	-
Log and Description Semantic Matching	Cisco	49,954/16,651/16,651	26,975/8,992/8,991	1,894/631/631
	Huawei	7,702/2,567/2,567	5,977/1,992/1,991	4,485/1,495/1,494
	H3C*	2,606/868/868	2,837/946/945	2,223/741/740
Log and Possible Cause Ranking	Huawei	3,851/1,283/1,283	3,097/1,032/1,032	2,361/787/787

* indicates logs not involved in the pre-training.

- **Fault Phenomenon Identification (FPI).** FPI is also a log-single task to identify the fault category to which the log belongs. Different from the previous failure identification task, FPI is a multi-label classification task due to a log may appear in more than one fault category.
  Dataset and Metric. We collect 602 real operational logs of Huawei switches from enterprise as the dataset, covering 43 failure categories, these logs are annotated by experts.Due to data privacy concerns, this dataset is currently not publicly available. Unlike the multi-class classification task, we report *Average Accuracy* of all samples [83] as the evaluation metric, where the accuracy for each sample is the number of correctly predicted labels.
- **Log and Description Semantic Matching (LDSM).** LDSM is a log-pair task aimed at determining whether the semantics of a given log align with the corresponding natural language description, where the input is a log and description pair, and the output is True or False.
  Dataset and Metric. We collect descriptions of logs from the documentation, then we build (log, description) pair as ground truth and randomly select one other description for each log as a negative sample. This task requires the model to accurately understand the semantics of the logs and descriptions. As a binary classification task, both positive and negative cases require attention, we report the *Accuracy* and *Weighed F1* as evaluation metrics.
- **Log and Possible Cause Ranking (LPCR).** LPCR is a log-pair ranking task to find the most probable answer from a list of possible causes for a given log. Specifically, the input is a log and an answer candidate set, then the model needs to find the most probable causes from the set and rank the candidate set based on the priority as the output.
  Dataset and Metric. We collect logs and the corresponding possible causes from the Huawei public documentation and build (log, possible cause) pairs as ground truth. Then

we randomly select 15 possible causes of other logs the ground truth as a candidate set. This task necessitates that the model understands the background information of the log to accurately identify its potential causes. As a typical ranking task, following [52], we report *Precision@K* and *Mean Reciprocal Rank* (MRR) as evaluation metrics, where MRR is a statistic measure for evaluating search algorithms.

D. Baselines

We categorize baselines for log understanding into five groups according to technology type: classical machine learning-based methods, traditional deep-learning methods, pre-trained language models, knowledge distillation methods, and large language models. In addition, to ensure a fair comparison, apart from LLMs, we reproduce all baselines from their repositories, and the parameters of baseline models are according to their original settings.

1) Classical Machine Learning Methods: K-nearest neighbor (KNN) is one of the most fundamental and simple classification methods [84]. In our implementation, we first store the token list (split from log messages) and the corresponding labels of the training log data. For inference, the prediction for a given log is determined by assigning it the label most frequently found among the nearest k training samples to the query point. Specifically, we obtain tokens list of the input log (excluding the header) by splitting the sequence based on spaces. Any token containing numeric characters is removed during preprocessing. For the sake of simplicity, following LightAD [81], we utilize the Jaccard index (also known as the Jaccard similarity coefficient) to find the nearest neighbors in the training set, defined as:

$$Jaccard(A, B) = \frac{|A \cap B|}{|A \cup B|}, \tag{8}$$

where A and B are the token sets of two log sequences. The coefficient ranges from 0 to 1, where 0 indicates no overlap between the sets (completely dissimilar) and 1 denotes identical sets (perfect similarity). And we set the number of neighbors k to 1 in our experiment. The performance of KNN on the LPCR task is excluded since KNN cannot be adapted to the ranking task.

2) Traditional Deep-Learning Methods: Traditional deep-learning methods convert each log message into a vector with the word embedding model and then input the vector to a deep neural network (CNN or BiLSTM) to analyze logs. Following previous works [85], [86], we choose the most widely used **BiLSTM** and **CNN** for log analysis as deep neural network models.

3) Pre-Trained Language Models: PLM-based log analysis methods follow the *pre-train and fine-tune* paradigm, where one pre-trained model serves as the backbone, and then is fine-tuned on different specific downstream tasks. We select one general pre-trained model (**BERT [24], [82]**) and two specific pre-trained models for logs (**Biglog [21]** and **KnowLog [22]**) as baselines. **Biglog** excels in capturing essential features due to its extensive training on the log corpus. In addition, **KnowLog** is the state-of-the-art log pre-trained model in log analysis, not only due to pre-training on log corpus but also enhancing the pre-training with documentation knowledge. Referring to KnowLog, we collect the descriptions of logs from the Huawei and Cisco public documentation as background knowledge to enhance the log pre-training in the same way. For fairness, we reproduce Biglog and KnowLog with the same pre-training setting on our log corpus.

4) Knowledge Distillation Methods: In the era of LLMs, the prevalent paradigm of data augmentation (DA) is to distill knowledge from LLMs, where it prompts the LLM to generate more data tailored to train the student model in a targeted skill or domain [44], [87]. This reasoning process-oriented distillation method is more effective compared to obtaining labels directly from the teacher model [45]. Considering the lack of research on knowledge distillation in the field of log analysis, we select Chain-of-Thought (COT)-based knowledge distillation as the baseline, which has proven to be an effective knowledge distillation method for the reasoning process in the field of NLP [46], [47].

Specifically, we select **T5 [88]** (*t5-base with 223M parameters*) as the student model, and GPT-4o as the teacher model. Given the specific log analysis task, we provide the inputs and labels from the training set to the teacher model, then utilize COT prompting to extract rationales from the teacher model. Finally, the student model is trained with the inputs and constructed rationales.

5) Large Language Models: LLM-based log analysis methods follow the *in-context learning* paradigm, where LLMs make predictions based on contexts augmented with a few examples without training. Following previous LLM-based log analysis works [26], [27], [28], [29], we select top-5 similar examples from the training set for each input log and use *bge-large-en-v1.5* as the embedding model to select examples, due to its excellent performance among all open-source embedding models. Considering the recency bias of LLMs [89], we arrange these examples in ascending order based on the cosine similarity. We select two proprietary LLMs (**ChatGPT** and **GPT-4o**) and one powerful open-source LLM (**LLama3-70B [78]**) as baselines. We utilize HTTP requests to invoke the OpenAI APIs and interact with proprietary LLMs. For open-source LLM, we deploy *Llama-3.3-70B-Instruct* in our environments to analyze logs.

In addition, since **COT** prompting [64] is verified to improve the performance of LLMs significantly. To verify the effectiveness of the MEC framework, we acquire knowledge with COT and then train the model with the same pre-training tasks for comparison. The specific prompt is: *You are an engineer in the maintenance and operation domain, Please help me understand this log, including parameters, description, possible causes and resolution procedures. Let's think step by step.*

E. Evaluation

We evaluate LUK by answering the following research questions (RQs):

- **RQ1: How effective is LUK compared with the current mainstream methods on downstream tasks?**

TABLE IV
RESULTS ON AD AND FI (SOFTWARE SYSTEM LOGS)

Methods	AD (Precision / Recall / F1) BGL	FI (Recall@1 / 2 / 3) OpenStack
KNN	98.37 / 96.80 / 97.58	90.00 / 95.00 / 95.00
CNN	**99.95** / 96.06 / 97.97	83.75 / 86.25 / 90.00
BiLSTM	**99.95** / 97.25 / 98.58	87.50 / 92.50 / 96.25
BERT	99.70 / 98.48 / 99.09	87.50 / 92.50 / **98.75**
Biglog	99.87 / 99.63 / 99.75	90.00 / 95.00 / **98.75**
KnowLog	99.58 / 98.69 / 99.13	82.50 / 92.50 / 96.25
T5-KD	99.87 / 99.56 / 99.91	86.25 / 92.50 / 98.75
LLama3-70B	46.87 / 95.23 / 71.05	85.00 / **96.25** / 96.25
ChatGPT	42.73 / 95.23 / 68.98	83.75 / 95.00 / 96.25
GPT-4o	46.85 / 95. 23 / 70.54	87.50 / **96.25** / 96.25
LUK (COT-ChatGPT)	99.95 / 99.87 / 99.91	91.25 / 95.00 / **98.75**
LUK (COT-GPT4o)	99.67 / **100.0** / 99.83	91.25 / 95.00 / **98.75**
LUK (MEC-LLama3)	99.87 / **100.0** / **99.93**	**92.50** / 95.00 / **98.75**
LUK (MEC-ChatGPT)	99.95 / 99.91 / 99.93	**92.50** / 95.00 / **98.75**
LUK (MEC-GPT4o)	99.87 / **100.0** / **99.93**	**92.50** / **96.25** / 96.25

In this RQ, we conduct a comprehensive evaluation of LUK in comparison to other state-of-the-art baselines on software systems and network device logs.

The experiment results are shown in Tables IV and VI. It is clear that apart from the LLMs-based baselines, LUK outperforms all other baselines. In addition, LUK with smaller parameters outperforms the LLMs on all tasks except for the MC and LDSM tasks. In particular, compared to the best results of baselines on the Failure Identification task of software system logs, LUK improves on Recall@1 by 2.5%. And on the Module Classification task of network device logs, LUK's accuracy improved by an average of 2.34%. This evidence underscores the superior performance of LUK in the domain of log analysis. We observe that the KNN algorithm excels specifically in anomaly detection and failure identification tasks, achieving results comparable to those of PLMs. However, its performance diminishes significantly in other knowledge-driven tasks. This performance can be attributed to the relatively simple anomaly patterns in public anomaly detection datasets [80], [81], which often contain limited anomaly patterns and numerous repetitive logs. The relatively simple anomaly patterns allow lexical pattern matching to suffice for basic detection tasks. Conversely, tasks that require a deeper understanding of log semantics reveal KNN's limitations, as it struggles to understand the information conveyed by log data. This inability to grasp semantic context restricts KNN's generalizability, making it less effective in more complex log analysis scenarios. In contrast, LUK not only excels in public anomaly detection datasets but also achieves state-of-the-art performance across a wider range of tasks, highlighting its advanced capabilities not only in pattern recognition typical of machine learning but also in the semantic understanding of logs. Further, traditional DL models exhibit relatively poor performance, primarily attributed to their limited capacity to capture the semantics of logs. When compared to BERT and Biglog, it becomes evident that pre-training on an extensive log corpus enhances the semantic understanding abilities of PLMs. However, a more in-depth analysis uncovers a notable challenge: the presence of domain-specific terminologies and the concise nature of logs act as barriers, and the lack of external knowledge limits these models from making further breakthroughs.

To verify the effectiveness of the expert knowledge obtained from the LLM, we compare it with the KnowLog, whose knowledge is obtained from the documentation. Our experimental findings demonstrate that MEC-based LUK outperforms KnowLog. More specifically, LUK (MEC-GPT4o) improves the average Accuracy on the Cisco and Huawei datasets regarding the MC and LDSM tasks by 2.38% and 1.15%, respectively. It is noteworthy that Huawei's documentation provides more detailed log descriptions compared to the relatively concise descriptions found in Cisco's documentation. This gap highlights the ability of LLM to generate expert knowledge of equal or better quality than high-quality documentation. Given the scarcity of documented descriptions for most logs in practice, obtaining expert knowledge through KnowLog becomes impractical. For instance, the challenge of sourcing relevant documents for logs from Loghub [38] significantly hinders the effectiveness of KnowLog. Our approach, operating independently of documentation, consistently yields superior results and reduces the reliance on human experts for complex knowledge construction, offering a more feasible and practical alternative to traditional documentation-based methods.

In comparison to the knowledge distillation-based method (T5-KD), the COT-based knowledge distillation approach exhibits suboptimal performance in specific log analysis tasks. This gap can be attributed to the unique nature of log analysis, which typically necessitates a deep understanding of domain-specific knowledge. The chain-of-thought methodology, with its sequential reasoning framework, may introduce noise or overlook critical expert knowledge, leading to diminished performance in tasks where precise understanding is crucial. In comparison, LUK prioritizes the construction of high-quality expert knowledge explicitly before fine-tuning it for specific tasks. By pre-building a solid foundation of domain-specific knowledge, LUK can better address the complex challenges in log analysis tasks.

In comparison to LLMs, our results show that apart from LDSM and LPCR tasks, three LLMs perform worse than the fine-tuned smaller PLMs. This difference can be attributed to the existence of a significant domain gap, which hampers the ability of LLMs to effectively address specialized tasks directly. Consequently, this limitation restricts their potential to fully exploit the wealth of knowledge they encapsulate. Hence, extracting knowledge from LLMs before reasoning directly bridges these critical gaps for task-specific analysis. Furthermore, our findings suggest that GPT4o-based enhancement outperforms the other two LLMs on most datasets, indicating the potential of leveraging stronger LLMs to significantly enhance the performance of smaller models in various tasks.

Comparing MEC with COT, MEC emerges as the superior performer. This superiority can be attributed to the collaborative essence of MEC, where multiple experts collaborate to improve the accuracy and comprehensiveness of the knowledge extracted from the LLM, effectively mitigating the problem

TABLE V
RESULTS ON MODULE CLASSIFICATION (MC) AND LOG AND DESCRIPTION SEMANTIC MATCHING (LDSM) TASKS. (NETWORK DEVICE LOGS)

Methods	MC (Accuracy / Weighted F1)				LDSM (Accuracy / Weighted F1)			
	Cisco		Huawei		Cisco		Huawei	
	Switches	Routers	Switches	Routers	Switches	Routers	Switches	Routers
KNN	42.77 / 42.86	46.75 / 46.41	67.71 / 66.64	62.84 / 60.90	53.00 / 52.65	51.45 / 50.83	50.33 / 49.73	47.56 / 46.88
CNN	56.89 / 56.85	57.46 / 54.92	74.52 / 73.95	72.78 / 72.23	84.04 / 84.04	80.99 / 80.99	86.05 / 86.05	82.37 / 82.30
BiLSTM	55.74 / 55.63	57.17 / 56.76	76.52 / 75.49	73.96 / 73.30	89.45 / 89.44	85.42 / 85.41	87.85 / 87.85	84.43 / 84.40
BERT	62.67 / 61.38	62.72 / 62.60	82.37 / 81.20	81.18 / 79.20	93.06 / 93.06	90.01 / 90.00	93.18 / 93.18	90.06 / 90.05
Biglog	62.69 / 62.76	63.15 / 61.17	83.24 / 83.25	82.36 / 81.19	93.32 / 93.32	91.46 / 91.46	94.19 / 94.19	93.62 / 93.61
KnowLog	63.33 / 63.35	63.65 / 63.12	84.11 / 83.65	83.55 / 82.41	95.05 / 95.05	92.78 / 92.78	97.15 / 97.15	96.43 / 96.43
T5-KD	62.96 / 62.36	63.86 / 61.01	83.60 / 83.23	83.55 / 82.82	95.29 / 95.28	93.20 / 93.20	95.25 / 95.25	93.97 / 93.97
LLama3-70B	61.61 / 60.95	62.99 / 60.55	82.98 / 82.83	81.66 / 80.34	84.48 / 84.31	88.32 / 88.24	95.49 / 95.49	95.57 / 95.57
ChatGPT	57.11 / 54.41	57.99 / 54.56	79.14 / 77.19	78.22 / 75.51	82.14 / 81.96	84.64 / 84.47	85.75 / 85.75	90.31 / 90.22
GPT-4o	62.12 / 61.62	63.86 / 61.01	83.25 / 82.80	81.30 / 79.87	89.04 / 88.99	93.12 / 93.11	97.23 / 97.23	**97.79 / 97.79**
LUK (COT-ChatGPT)	63.49 / 63.72	64.51 / 63.33	83.42 / 82.70	83.19 / 82.34	94.93 / 94.92	93.11 / 93.11	96.33 / 96.33	95.73 / 95.73
LUK (COT-GPT4o)	63.47 / 63.51	63.61 / 63.02	83.60 / 83.20	82.84 / 81.92	94.92 / 94.91	93.18 / 93.18	96.46 / 96.46	95.93 / 95.93
LUK (MEC-LLama3)	65.67 / 65.27	66.67 / 65.12	85.25 / 84.95	85.21 / 84.01	96.45 / 96.45	94.13 / 94.13	97.66 / 97.66	97.39 / 97.39
LUK (MEC-ChatGPT)	65.54 / 65.12	66.29 / 64.27	84.64 / 84.47	85.09 / 83.82	96.32 / 96.32	93.80 / 93.80	96.92 / 96.92	96.74 / 96.74
LUK (MEC-GPT4o)	**66.16 / 66.02**	**67.20 / 65.41**	**85.51 / 85.26**	**85.33 / 84.73**	**96.63 / 96.63**	**94.32 / 94.32**	**97.66 / 97.66**	97.34 / 97.34

TABLE VI
RESULTS ON LPCR AND FPI (NETWORK DEVICE LOGS). NOTE: THE RESULTS OF PROPRIETARY LLMS ON THE FPI TASK CANNOT BE COMPARED DUE TO DATA PRIVACY ISSUES. THE RESULTS OF KNN ON THE LPCR TASK CANNOT BE COMPARED DUE TO KNN'S INABILITY TO ADAPT TO RANKING TASKS

Methods	LPCR (Precision@1 / 3 / MRR)		FPI (Accuracy)
	Huawei		Huawei
	Switches	Routers	Switches
KNN	-	-	76.50
CNN	54.30 / 77.26 / 67.99	53.45 / 75.77 / 67.35	69.00
BiLSTM	59.27 / 78.04 / 71.22	51.45 / 69.56 / 63.76	74.83
BERT	76.18 / 91.54 / 84.70	72.57 / 91.59 / 82.61	88.75
Biglog	83.33 / 94.02 / 89.38	82.98 / 95.59 / 89.49	89.58
KnowLog	87.50 / 93.01 / 91.84	89.68 / 96.59 / 93.99	94.17
T5-KD	86.11 / 93.56 / 91.41	84.48 / 92.59 / 90.33	-
LLama3-70B	**95.61 / 98.04 / 97.27**	**97.48 / 99.49 / 98.52**	88.83
ChatGPT	90.95 / 95.11 / 93.88	97.20 / 98.34 / 98.19	-
GPT-4o	95.65 / 98.29 / 97.27	97.39 / 99.09 / 98.47	-
LUK (COT-ChatGPT)	85.18 / 95.03 / 90.55	84.58 / 95.99 / 90.64	90.42
LUK (COT-GPT4o)	86.34 / 93.79 / 91.37	87.88 / 95.19 / 92.75	92.50
LUK (MEC-LLama3)	90.45 / 95.81 / 94.06	90.49 / 96.99 / 94.52	95.83
LUK (MEC-ChatGPT)	89.37 / 95.19 / 93.43	88.48 / 94.69 / 90.33	93.33
LUK (MEC-GPT4o)	90.92 / 96.04 / 94.46	90.69 / 96.09 / 94.39	**96.67**

of inaccurate and incomplete knowledge due to hallucinations. Moreover, the MEC framework showcases its effectiveness in generating comprehensive and accurate insights through the iterative feedback and refinement mechanism.

In conclusion, the superior performance of LUK in log analysis tasks underscores its effectiveness across various datasets. By leveraging expert knowledge extracted from LLMs, LUK demonstrates the advantage of enhancing smaller pre-trained models with professional insights. This approach not only optimizes the utilization of the rich knowledge within LLMs but also introduces a new paradigm in the realm of log analysis, paving the way for more efficient and insightful methodologies. In addition, LUK's process of acquiring knowledge is automated and independent of experts, making it more practical.

- **RQ2: How effective is LUK in generalization ability?**

As the system evolves, many previously unseen logs will be collected [85], and it is unrealistic to collect all the real-world logs for pre-training. In this RQ, to verify the generalization of LUK, we conduct experiments on downstream tasks where the logs in these datasets do not appear in the pre-training phase. Specifically, for software logs, we collect ThunderBird and Spirit from Loghub [38] as the datasets of the Anomaly Detection task. For network device logs, as shown in Table III, we collect logs from other vendors (H3C) and devices (Security) as the datasets.

The experimental results are shown in Table VII. It can be found that LUK still has superior performance on the pre-training unseen logs, especially on the Module Classification task, where LUK improves 3.38% on average Accuracy compared to T5-KD. These results suggest that the expert knowledge distilled from LLMs plays a key role in empowering smaller PLMs to develop a deeper understanding of log. This proficiency ensures that the PLM is trained without the tendency to overfit, which would prevent fine-tuned models from effectively analyzing unseen logs because they fail to grasp crucial information essential for accurate log analysis.

In comparison to KnowLog, LUK still has a clear advantage over KnowLog, which derives knowledge from documentation. The expert knowledge derived from LLMs offers a deeper level of comprehension and learning, enabling the PLM to generalize more effectively to new logs. Conversely, KnowLog, constrained by information within documentation, may not provide profound insights to support robust generalization of the PLM to unseen logs.

In essence, our findings affirm that LUK excels in its generalization capabilities by analyzing unseen logs. The infusion of expert knowledge sourced from LLMs equips the smaller PLM with the ability to leverage crucial insights from expert knowledge, thereby avoiding overfitting and improving its performance in log analysis tasks. This underscores the pivotal role

TABLE VII
RESULTS OF GENERALIZATION ABILITY EXPERIMENTS (SOFTWARE SYSTEM LOGS)

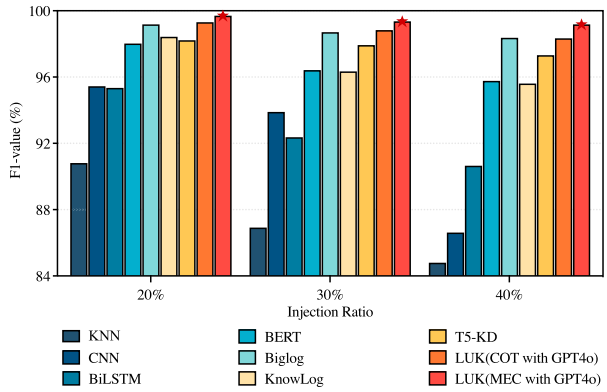
Methods	Anomaly Detection (Precision / Recall / F1)	
	ThunderBird	Spirit
KNN	99.08 / 99.89 / **98.98**	96.86 / 99.55 / 98.18
CNN	94.36 / 99.97 / 97.09	97.55 / 97.11 / 97.33
BiLSTM	98.10 / 96.96 / 97.52	98.78 / 95.61 / 97.17
BERT	98.35 / 97.21 / 97.78	**99.98** / 98.74 / 99.36
Biglog	97.04 / 96.33 / 96.68	99.56 / 99.07 / 99.31
KnowLog	99.56 / 94.46 / 96.95	99.38 / 99.20 / 99.29
T5-KD	99.35 / 95.08 / 97.17	98.91 / 99.63 / 99.27
LLama3-70B	55.05 / 93.96 / 69.42	78.16 / 97.49 / 86.76
ChatGPT	48.21 / 93.10 / 63.52	79.19 / 98.20 / 87.68
GPT-4o	49.89 / **100.0** / 66.57	75.44 / 90.32 / 82.21
LUK (COT-ChatGPT)	98.55 / 95.06 / 96.77	99.81 / 98.92 / 99.37
LUK (COT-GPT4o)	99.35 / 95.08 / 97.17	98.92 / **99.81** / 99.37
LUK (MEC-LLama3)	**99.78** / 96.72 / 98.23	99.09 / 99.81 / 99.45
LUK (MEC-ChatGPT)	99.57 / 96.51 / 98.02	99.89 / 99.16 / 99.52
LUK (MEC-GPT4o)	99.57 / 96.92 / 98.23	99.27 / **99.81** / **99.54**

of expert knowledge from LLMs in improving the adaptability of LUK to navigate the complex log analysis.

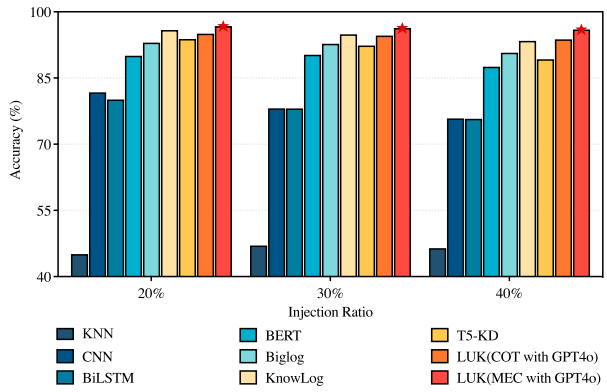
- **RQ3: How effective is LUK on unstable log data?**

Real-world systems logs are *unstable*, meaning that new but similar logs often appear, which is caused by the fact that developers may frequently modify the logging statements in source code. According to the investigation [90], around 20% - 45% of logging statements may change throughout the lifetime. In this RQ, to evaluate the effectiveness of LUK on unstable logs, we conduct experiments on two log analysis tasks with unstable logs. Following previous experimental setups [85], we conduct experiments on two distinct log analysis tasks, which are based on the BGL dataset of Anomaly Detection and Huawei-Switches dataset of Log and Description Semantic Matching. Both tasks utilize synthetic datasets specifically constructed to simulate the unstable characteristics of real-world logs. The BGL dataset has a limited number of log templates, and anomaly detection on this dataset can often achieve strong performance using simple keyword-based matching techniques [81]. At the same time, the LDSM task emphasizes a deeper understanding of log semantics. To more deeply evaluate the model's ability to understand the semantics of logs, rather than relying on specific patterns to analyze logs, we synthesize datasets for both tasks by introducing controlled noise into the original logs.

To simulate log instability, we apply two transformation rules to the original logs while maintaining their semantic integrity. These transformations are inspired by the common practices developers follow when updating logging statements, such as modifying or removing certain words. As shown in Fig. 8, the first strategy is to remove the stop words, where we randomly remove from the original log messages; the second strategy is to replace the synonyms, where we replace certain words in the original log messages with their synonyms based on WordNet.



(a) Anomaly Detection (BGL)



(b) LDSM (Huawei Switches)

Fig. 7. Results on different synthetic datasets of unstable log.

To ensure the robustness of our evaluation, we inject noise into the original logs by randomly applying these two strategies at specific proportions. Importantly, these transformations preserve the semantic meaning of the logs, ensuring that the anomaly labels or semantic relationships in the datasets remain unaffected.

The experimental results are shown in Fig. 7, it can be seen that LUK performs much better than other baselines. With the increasing injection ratio of unstable logs, the performance of all methods has declined in different degrees. However, MEC-based LUK declines relatively smoothly and still maintains high performance even under a high injection ratio. Specifically, as the injection ratio increased from 20% to 40%, LUK's F1 and Accuracy decreased by merely 0.54% and 0.78% on the anomaly detection and LDSM tasks, respectively. It confirms that LUK is robust enough to the unstable logs. The reason is that incorporating expert knowledge from LLMs into the smaller model assists in noise filtering, thereby enhancing the accuracy of log analysis for the smaller pre-trained model. Compared with KNN, the results of anomaly detection demonstrate that KNN's performance degrades significantly as the noise ratio increases. On the LDSM task, KNN's accuracy is consistently below 50%, indicating that KNN's reliance on lexical similarity prevents it from effectively understanding log semantics. KNN's effectiveness in anomaly detection stems from the high degree of repetition in the original dataset, where

TABLE VIII
RESULTS OF GENERALIZATION ABILITY EXPERIMENTS (NETWORK DEVICE LOGS)

Methods	Module Classifiction		Log and Description Semantic Matching					LPCR
	H3C Switches	H3C Routers	Cisco Security	Huawei Security	H3C Switches	H3C Routers	Security	Huawei Security
KNN	61.74 / 59.07	64.86 / 62.36	50.55 / 50.19	50.06 / 49.10	61.05 / 60.70	53.75 / 53.64	57.02 / 56.62	-
CNN	69.49 / 67.55	70.72 / 69.71	70.36 / 70.36	81.73 / 81.73	83.29 / 83.19	83.60 / 83.59	82.30 / 82.30	56.05 / 79.07 / 69.95
BiLSTM	70.21 / 68.45	71.40 / 69.93	74.01 / 74.01	79.12 / 79.12	80.88 / 80.83	83.81 / 83.80	82.57 / 82.57	55.65 / 79.21 / 69.80
BERT	81.11 / 79.78	77.93 / 76.05	79.08 / 79.08	89.22 / 89.22	87.44 / 87.41	88.25 / 88.25	85.94 / 85.94	67.89 / 89.73 / 79.55
Biglog	81.59 / 79.53	79.50 / 77.80	82.73 / 82.69	94.19 / 94.19	89.40 / 89.37	91.11 / 91.11	92.02 / 92.02	75.78 / 92.23 / 84.59
KnowLog	81.60 / 80.27	79.50 / 79.22	84.94 / 84.93	96.45 / 96.45	93.78 / 93.78	91.85 / 91.85	93.65 / 93.65	83.94 / 91.71 / 89.77
T5-KD	82.32 / 81.26	80.41 / 80.15	84.63 / 84.63	95.31 / 95.31	92.86 / 92.86	94.50 / 94.49	93.78 / 93.78	84.86 / 91.57 / 90.19
LLama3-70B	79.90 / 78.35	79.28 / 78.22	89.42 / 89.42	96.98 / 96.98	94.58 / 94.57	96.08 / 96.08	95.67 / 95.67	**96.97 / 98.94 / 98.11**
ChatGPT	77.48 / 73.77	76.35 / 74.10	85.42 / 85.38	90.87 / 90.76	91.01 / 90.96	91.96 / 91.90	93.24 / 93.24	94.95 / 97.01 / 96.56
GPT-4o	79.90 / 78.35	79.95 / 79.20	**93.43 / 93.42**	**98.49 / 98.49**	96.43 / 96.43	**97.56 / 97.56**	96.49 / 96.49	96.44 / 98.28 / 97.76
LUK (COT-ChatGPT)	83.05 / 81.52	80.63 / 79.00	84.31 / 84.30	96.33 / 96.33	93.89 / 93.89	94.18 / 94.18	93.91 / 93.91	82.23 / 94.73 / 88.83
LUK (COT-GPT4o)	83.05 / 82.18	80.41 / 82.29	84.94 / 84.94	96.52 / 96.51	94.70 / 94.70	94.81 / 94.81	93.92 / 93.92	83.81 / 90.78 / 89.48
LUK (MEC-LLama3)	84.50 / 83.33	83.33 / 81.71	87.64 / 87.63	97.39 / 97.39	95.97 / 95.97	96.61 / 96.61	96.22 / 96.21	87.50 / 94.07 / 92.34
LUK (MEC-ChatGPT)	83.78 / 82.69	83.11 / 81.40	88.90 / 88.90	96.99 / 96.99	95.16 / 95.16	95.34 / 95.34	95.54 / 95.54	86.71 / 93.55 / 91.75
LUK (MEC-GPT4o)	**85.47 / 84.57**	**84.01 / 82.29**	89.06 / 89.06	97.52 / 97.52	**96.54 / 96.54**	96.93 / 96.93	**96.89 / 96.89**	87.36 / 94.47 / 92.22

a large portion of logs are identical or nearly identical. In such cases, KNN can rely on simple character or word matching to identify patterns. However, when noise is introduced, such as through stopword removal or synonym substitution, the repetitive patterns are disrupted, and KNN fails to generalize. This highlights KNN's inability to analyze semantic relationships, which becomes increasingly important in complex and diverse log data. To further investigate the characteristics of the anomaly detection datasets, we calculate the Jaccard index between logs in the training and testing sets. As shown in Fig. 9, the Jaccard index for these public datasets is concentrated around 1.0, indicating a high degree of overlap between the training and testing data. This overlap leads to data leakage, where KNN performs well simply by finding repetitive patterns in the dataset. However, as noise is injected, the Jaccard index drops to approximately 0.5, reducing the repetition in the dataset. Correspondingly, KNN's performance declines significantly, demonstrating its reliance on repetitive logs. This phenomenon has also been observed in prior studies, where *Yu* et al. [81] reports that more than 50% of the BGL dataset consists of duplicate logs, with other public datasets such as Thunderbird and Spirit exhibiting even higher duplication rates, exceeding 99%. These findings confirm that KNN and similar classical methods are heavily dependent on the presence of repetitive patterns. Considering that it is only effective when anomalies exhibit highly repetitive, template-aligned features, it lacks robustness and generalization capabilities when applied to more complex and diverse log data.

Compared with CNN and BiLSTM, traditional methods perform the worst on unstable logs, which suggests that the limited semantic understanding of traditional methods hinders their ability to analyze logs more robustly. Compared with BERT and Biglog, although pre-training on log corpus can further improve log understanding, models are still limited to understanding logs with professional knowledge. In contrast, KnowLog's reliance on knowledge solely extracted from the documentation

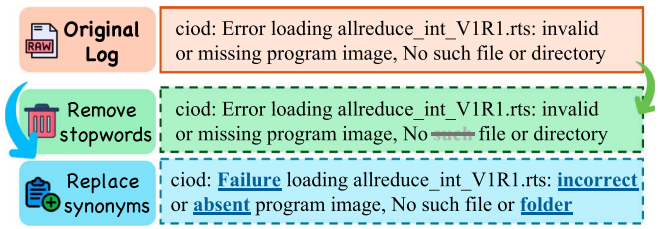


Fig. 8. Examples of synthesized logs.

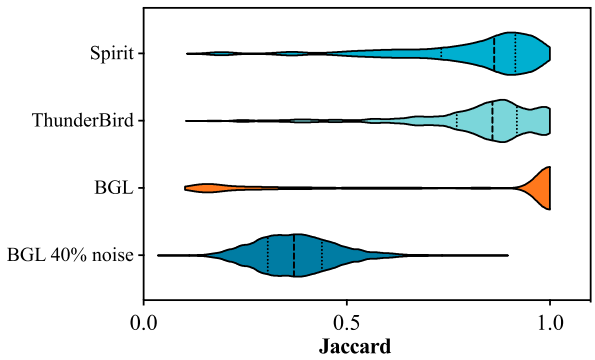
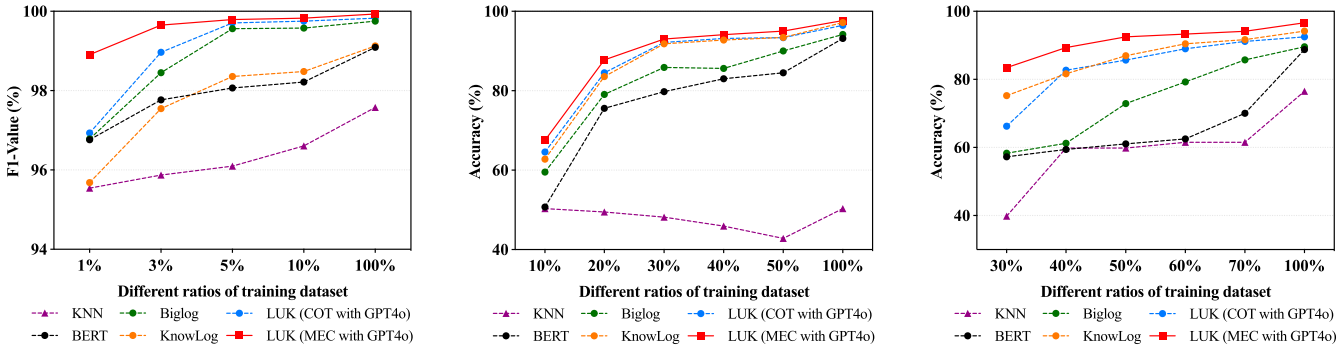


Fig. 9. Jaccard index on the anomaly detection datasets.

may lead to limitations in handling the evolving nature of log data. MEC-based LUK, with its multi-expert collaboration, outperforms KnowLog by providing deeper and adaptable expert knowledge for robust log analysis tasks. When compared to the COT-based methods, including T5-KD and COT-based LUK, MEC-based LUK showcases superior robustness on unstable logs. The collaborative essence of MEC, by incorporating multiple expert perspectives and iterative feedback loops, ensures the refinement and validation of expert knowledge for log analysis tasks, enhancing robustness and accuracy.



(a) Anomaly Detection (BGL) (b) LDSM (Huawei-Switches) (c) Fault Phenomenon Identification

Fig. 10. Results on different ratios of the training dataset.

Consequently, we can conclude that LUK showcases remarkable robustness when confronted with unstable logs, demonstrating that leveraging expert knowledge from LLMs equips smaller PLMs with the necessary insights to understand the semantics of logs, thereby enhancing their resilience against noise and instability.

- **RQ4: How effective is LUK with limited labeled logs?**

In real-world applications, obtaining a substantial amount of annotated samples is challenging, as noted by prior studies [12], [23]. This poses a significant barrier to the performance of automated log analysis models. To evaluate the effectiveness of LUK in low-resource scenarios, where annotations are scarce, we conduct experiments on Anomaly Detection, Log and Description Semantic Matching, and Fault Phenomenon Identification with different ratios of training datasets. Both AD and LDSM tasks possess a relatively high volume of labeled data in these evaluation tasks. However, considering the high cost of acquiring extensive annotations in real-world scenarios, it is critical to assess their performance under low-resource scenarios. Additionally, the FPI task is derived from real-world data, where the availability of labeled samples is inherently limited. Thus, evaluating FPI in low-resource settings aligns closely with practical constraints.

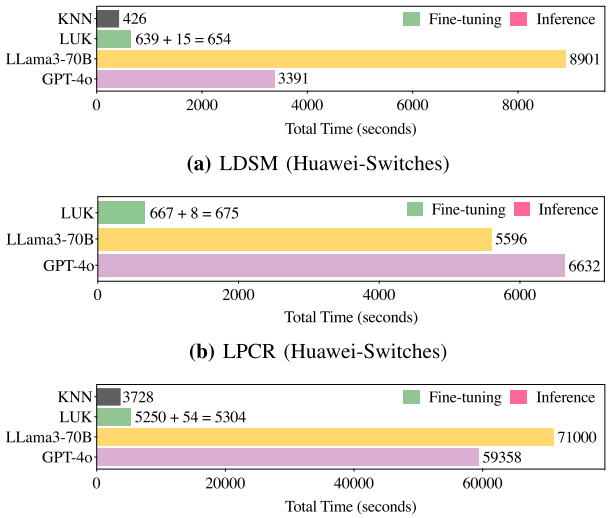
The experimental results are shown in Fig. 10. Considering that knowledge distillation based on fewer samples is difficult to train, we do not compare with T5-KD. We find that with the reduction of training samples, the performance of various models exhibits a decline, with BERT and Biglog demonstrating a particularly significant downward trend in the LDSM and FPI tasks, and KnowLog showing a significant decrease on the AD task. On the other hand, MEC-based LUK achieves optimal results by fine-tuning the models with different proportions of annotated samples. Specifically, on the AD task, compared to the full data, LUK drops only 1.03% in the F1-value with 1% of the training data. And on the FPI task, LUK's Accuracy drops by only 13.2% with 30% of the training data, while BERT and Biglog drop by 31.45% and 31.25%, respectively. This demonstrates that LUK gains expert knowledge from LLM, which helps to compensate for the shortcomings of the smaller pre-trained model when less annotated data is available. Models with restricted knowledge struggle to excel in scenarios with limited resources, highlighting the significance of leveraging knowledge to mitigate the need for extensive annotation and enhance performance in data-scarce environments. KnowLog primarily relies on knowledge extracted from documentation, which may be limited in scope and quality, especially in low-resource settings. In contrast, this proves that knowledge acquired by the LLM has a wider scope and can provide a more comprehensive and diverse range of information, rather than being limited to some specific documentation. In comparison to KNN, we observe that the performance of KNN declines significantly as the proportion of the training dataset decreases. This decline stems from KNN's reliance on the number of reference samples. With fewer labeled logs, KNN struggles to make accurate predictions, particularly for new or unseen logs. This highlights a fundamental limitation of KNN: its lack of understanding of the underlying semantics of logs. By relying solely on fixed patterns, KNN demonstrates poor generalization capabilities in low-resource scenarios. Compared with utilizing COT to acquire knowledge, on the FPI task with 30% of the training data, the Accuracy of MEC-based LUK is 17.2% higher than COT. This suggests that the MEC framework is more effective, which can be inferred that more rational and accurate knowledge enables the model to gain an advantage in low-resource scenarios.

It is worth noting that the FPI task is a real scenario dataset, which cannot be analyzed directly with LLMs due to privacy issues. By using log templates to obtain expert knowledge from LLMs, a task-specific model is constructed based on LUK, which proves the effectiveness of LUK and improves the efficiency of task analysis.

In summary, LUK achieves outstanding performance in low-resource scenarios, which stems from its ability to leverage expert knowledge acquired from LLMs, effectively compensating for smaller PLMs when annotated data availability is limited. By harnessing this expert knowledge properly, LUK enables the development of efficient and accurate models even in resource-constrained environments.

- **RQ5: How efficient is LUK in inference compared with LLMs?**

Efficiency is a critical factor in log analysis for practical applications, given the immense volumes of logs generated in



(a) LDSM (Huawei-Switches)

(b) LPCR (Huawei-Switches)

(c) Anomaly Detection (BGL)

Fig. 11. Time expense of different models during reasoning.

real-world systems [23]. In this RQ, we compare LUK's deployment and time costs with LLMs in inference. Specifically, to cover log analysis tasks for different input types, we perform experiments on Anomaly Detection, Log and Description Semantic Matching, and Log and Possible Cause Ranking three tasks to calculate the inference time with all testing samples. For fairness, we deploy LLama3-70B and LUK on the same hardware setup, consisting of 2 NVIDIA A100 GPUs, with a batch size of 1 for each model. It's worth noting that deploying LUK requires about 2 GB of GPU memory, while deploying LLama3-70B requires 150 GB of GPU memory. For GPT-4o, we utilize HTTP requests to invoke the APIs, then calculate the response time. Additionally, we leverage vLLM[4] to accelerate the inference process on LLama3. For further comparison, we include a lightweight kNN baseline, which operates on CPUs without neural network computations by calculating the Jaccard index between tokenized inputs.

The results are shown in Fig. 11. We notice that LUK is significantly faster than GPT-4o and LLama3. Specifically, excluding the fine-tuning time, the average inference speed of LUK is 718x and 868x faster than GPT-4o and LLama3-70B, respectively. Even considering the fine-tuning time, the average inference speed of LUK is still 9x and 11x faster than GPT-4o and LLama3-70B. In addition, the average response time for each input is 4.13 and 4.69 seconds for GPT-4o and LLama3-70B, respectively, while LUK's average response time is 0.0058 seconds. This swift response time positions LUK as a more practical solution for real-world log analysis scenarios. This efficiency can be attributed to the fact that LLMs possess extensive parameter sizes and generate outputs sequentially, necessitating more computational steps to deliver results. In contrast, LUK, with its smaller parameter sizes and more professional log analysis capabilities, can swiftly address specific tasks following knowledge enhancement and fine-tuning.

[4]https://github.com/vllm-project/vllm

TABLE IX
ABLATION STUDIES FOR LUK, WHERE TP AND SA REPRESENT THE PRE-TRAINING TASKS TOKEN PREDICTION, AND SEMANTIC ALIGNMENT FOR LUK, RESPECTIVELY. MLM REPRESENTS THE PRE-TRAINING TASK MASKED LANGUAGE MODELING OF BERT, AND AP REPRESENTS THE PRE-TRAINING TASK ABBREVIATION PREDICTION OF KNOWLOG

Methods	MC (Accuracy / Weighted-F1)		LDSM (Accuracy / Weighted-F1)	
	Huawei		Huawei	
	Switches	Routers	Switches	Routers
BERT	82.37 / 81.20	81.18 / 79.20	93.18 / 93.18	90.06 / 90.05
LUK (MEC-GPT4o)	**85.51 / 85.26**	**85.33 / 84.73**	**97.66 / 97.66**	**97.34 / 97.34**
⊢ w/o Evaluator	83.77 / 83.26	82.30 / 81.08	96.57 / 96.57	95.53 / 95.53
⊢ w/o Evaluation Examples	84.38 / 84.17	82.13 / 81.06	96.64 / 96.64	95.63 / 95.63
⊢ w/o TP	83.25 / 82.65	81.66 / 79.92	95.95 / 95.95	95.68 / 95.68
⊢ w/o SA	83.77 / 83.19	84.14 / 82.92	95.25 / 95.25	94.68 / 94.68
⊢ only with MLM	83.24 / 83.25	82.36 / 81.19	94.19 / 94.19	93.62 / 93.61
⊢ only with AP	83.60 / 82.89	82.72 / 81.68	94.39 / 94.39	93.97 / 93.97

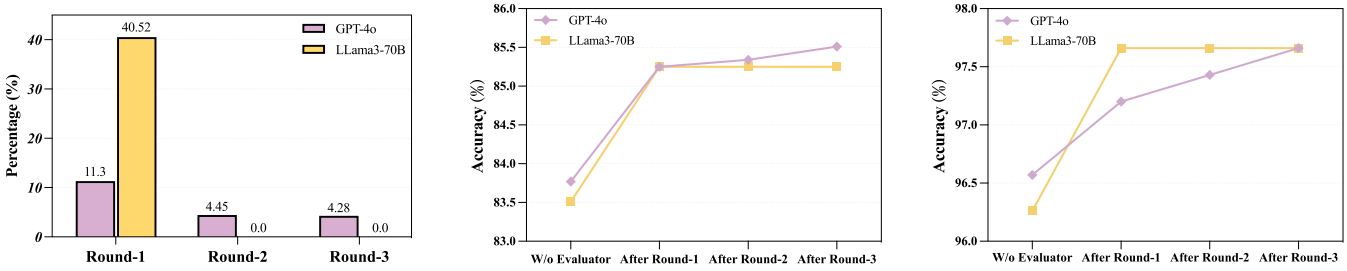
While KNN demonstrates certain efficiency advantages in log analysis(e.g., achieving 1.4x faster inference than LUK in the anomaly detection task), it lacks the robustness and generalizability required for complex and diverse log analysis systems. Efficiency alone is insufficient for building a comprehensive log analysis framework. By distilling knowledge from LLMs into smaller models, LUK strikes a balance between computational efficiency and semantic understanding, allowing it to harness the advanced capabilities of LLMs while maintaining practical inference times. In the future, we will further explore enhancements on more lightweight pre-trained models to further improve the efficiency of LUK.

In conclusion, LLMs demand higher deployment resources and exhibit slower inference speeds. On the other hand, the knowledge-enhanced LUK offers rapid responses to inputs with limited deployment resources, making it a highly recommended choice for executing specific tasks in real-world scenarios. This advantage underscores the practicality and efficiency of LUK in comparison to traditional LLMs, such as GPT-4o and LLama3-70B.

F. Ablation Studies

To verify the effectiveness of the multi-expert collaboration framework and knowledge-enhanced pre-training tasks in LUK, we perform ablation experiments on MC and LDSM, which are two typical tasks of multi-class classification and semantic matching. The results are shown in Table IX, where we notice that: Overall, LUK achieves optimal performance with the complete modules. The absence of any module can lead to performance degradation, which proves that MEC and pre-training tasks contribute positively.

Further, in our multi-expert collaborative framework, the *Evaluator* plays a crucial role in evaluating the content generated by the *Executor* to ensure the completeness and accuracy of the output expert knowledge. We remove the *Evaluator* role and directly utilize the content generated by the *Executor*. The results indicated a performance decrease in the absence of the *Evaluator*, emphasizing the pivotal role of feedback from the *Evaluator* in producing high-quality knowledge and reducing errors. In addition, to verify the effectiveness of the contrastive examples for the *Evaluator*, we conduct an ablation analysis



(a) Proportion of unsatisfactory knowledge **(b)** MC (Huawei-Switches) **(c)** LDSM (Huawei-Switches)

Fig. 12. Figure (a) shows the proportion of unsatisfactory knowledge in each iteration. Figures (b) and (c) show the results of the refined knowledge based on the evaluation feedback each time.

by removing the referenced examples for the *Evaluator* and requiring evaluations to be conducted in the reference-free setting. The experimental results reveal a substantial decline in the performance of the LUK when lacking referenced examples, even surpassing the negative impact observed when the *Evaluator* role was entirely removed. This outcome suggests that the lack of referenced examples deprives the evaluation process of essential guidance and benchmarks, increasing the quality evaluation challenges faced by the system.

To verify the performance of the iterative evaluation and refinement mechanism, we first statistics the proportion of unsatisfactory knowledge in each iteration, and then verify the performance of the refined knowledge based on the evaluation feedback each time on the MC and LDSM tasks, where we set up 3 iterations in our experiment. As shown in Fig. 12, we can find a consistent decline in the proportion of unsatisfactory knowledge with each successive iteration. Simultaneously, the performance of the enhanced model exhibits steady improvement, underscoring the positive impact of the iterative evaluation and refinement mechanism within the framework. Notably, our observations indicate that a significant portion of knowledge refinement occurs following the initial feedback round, with subsequent iterations primarily focusing on refining a smaller fraction of the knowledge. To save costs, we suggest that the framework can iterate once as a cost-effective strategy, effectively balancing performance gains with resource optimization.

For pre-training tasks, the performance of the model declines without either token prediction (TP) or semantic alignment (SA) pre-training task, which suggests that pre-training on logs with knowledge enhancement helps the model capture knowledge from external information and improves log understanding. Specifically, without the TP task, the model drops more significantly on the MC task, which implies that understanding domain tokens can enhance log understanding. And lacking the SA task, the model drops more remarkably on the LDSM task, which implies that external knowledge can enrich the contextual information of logs. To further verify our proposed word-level Token Prediction (TP) task, we also compare it with other word-level pre-training tasks, including BERT's Masked Language Modeling (MLM) task and KnowLog's Abbreviation Prediction (AP) task. The results demonstrate the superior capability of our token prediction task in integrating expert knowledge effectively. Unlike MLM and Abbreviation Prediction tasks, our proposed pre-training task requires models to not only predict tokens within the log itself but also dynamically capture contextual insights from domain-specific knowledge with the knowledge perception module. This distinctive aspect sets our approach apart, as BERT solely relies on internal information and cannot leverage external knowledge. Conversely, KnowLog is limited in its scope, focusing solely on abbreviations and failing to comprehend other critical log elements such as parameters, conditions, and more, which are essential for a comprehensive understanding of domain-specific terminology.

V. Discussions

A. Qualitative Analysis

To show the usefulness of LUK more intuitively, we select two representative cases for qualitative analysis. Specifically, we employ a pre-trained model to obtain embedding representations of the input log and natural language (NL) description, then calculate cosine similarity as the score to demonstrate their capability of semantic understanding. As shown in Table X, to provide a challenge in this case, we deliberately select logs with closer events and then compute their similarity to the description, which is matched only with one log.

From this case, we notice that, for the matched example, BERT demonstrates better performance, whereas Biglog's performance is comparatively weaker when compared to BERT. This suggests that only pre-training on logs makes it challenging to capture log semantics adequately without external knowledge, and this may enlarge the semantic differences between logs and natural language. In contrast, LUK achieves the highest similarity score on the matched example and the lowest similarity score on the unmatched example, which suggests that, benefiting from expert knowledge, LUK exhibits superior proficiency in capturing log semantics for log understanding. It also verifies the effectiveness of knowledge acquisition from LLMs. For the unmatched example, both BERT and Biglog exhibit significantly weaker performance compared to LUK, which implies that BERT and Biglog have limitations in recognizing log semantics. These results reveal the remarkable performance of LUK in log understanding.

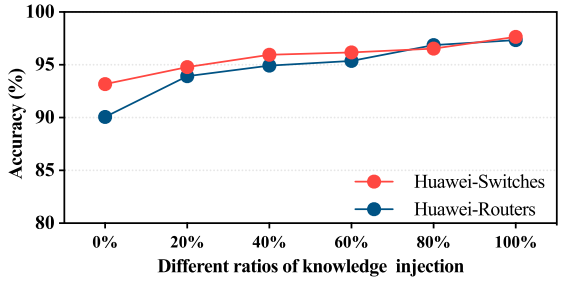
TABLE X
QUALITATIVE EXAMPLES OF LUK AND BASELINES. WE CALCULATE THE COSINE SIMILARITY AS THE SCORE FOR ANALYSIS

Label	Index	Examples	Models	Score
Match	1	**Log:** OPSA/3/OPS_CLI_CLOSE_FAIL: Failed to stop the terminal using the script. (Script=[script-name], event=[event-name], instance=[instance-id],terminal=[cli-id]) **NL Description:** There was a failure in stopping the terminal using a specific script.	BERT Biglog LUK	0.671 0.624 **0.740**
	2	**Log:** SYSTEM/4/SYS_IMAGE_ERROR: The next startup image package is error. (imageIndex=[imageIndex], curImageName=[curImageName], nextImageName=[nextImageName], errReason=[errReason]) **NL Description:** An error occurred in the next startup image package.	BERT Biglog LUK	0.723 0.629 **0.788**
UnMatch	1	**Log:** OPSA/3/OPS_TERMINAL_WRITE_FAIL: Failed to display the string on the terminal using the script. (Script=xx,event=xx, instance=xx, string=xxx, terminal=xx) **NL Description:** There was a failure in stopping the terminal using a specific script.	BERT Biglog LUK	0.657 0.646 **0.514**
	2	**Log:** TRILL/4/TRILL_RECV_ERR_PKT: TRILL-INFO: Drop error packet. (PktType=[PktType], ProcessId=[ProcessId], ErrReason=[ErrReason], ErrCount=[ErrCount], InterfaceName=[InterfaceName]) **NL Description:** An error occurred in the next startup image package.	BERT Biglog LUK	0.671 0.486 **0.077**

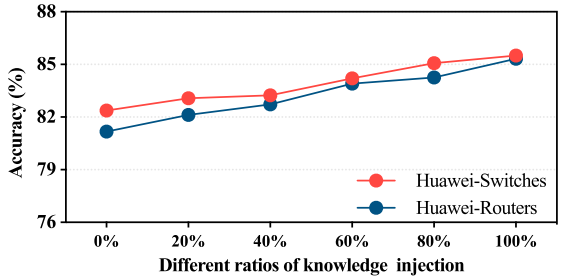
B. Cost Analysis

To quantitatively evaluate the cost-effectiveness of leveraging LLMs for expert knowledge construction, we conduct a detailed cost analysis using GPT-4o[5] to build the multi-expert collaboration framework. The average API cost for constructing expert knowledge per log entry is **$0.07**. Considering the high redundancy in log data (typically containing millions of entries), we recommend applying log parsing techniques to deduplicate logs by retaining only one instance per unique template. This significantly reduces the cost of domain knowledge construction. For instance, for a deduplicated dataset containing 10,000 logs, the total knowledge construction would cost only $700. This approach demonstrates significant cost advantages compared to conventional LLM-based log analysis. Analyzing each log entry directly with ICL-based GPT-4o would cost about $0.02 per entry, which becomes prohibitively expensive when scaling to millions of logs.

To explore the trade-off between the scale of expert knowledge and model performance, we conduct experiments using knowledge sampled from Huawei logs at different proportions. The enhanced models are evaluated on the MC and LDSM tasks, where the granularity of the domain knowledge they need is more representative. As shown in Fig. 13, the performance of the models improves with increased knowledge injection ratio, plateauing beyond a 60% knowledge injection ratio. This indicates that collecting a sufficient but not exhaustive amount of domain knowledge is effective for enhancing the models, thereby further reducing the cost of knowledge construction. These findings demonstrate that LUK effectively balances performance and cost by minimizing the reliance on LLM APIs and optimizing the scale of domain knowledge used for pre-training.



(a) Log and Description Semantic Matching



(b) Module Classification

Fig. 13. Performance with different proportions of knowledge injection.

C. Case Analysis

1) Evaluation and Refinement Mechanism of the MEC Framework: To intuitively understand the workflow of the MEC framework and analyze the impact of the evaluation and refinement mechanism on knowledge generation, we provide a specific case of expert knowledge construction as shown in Fig. 14, where this input log from the Huawei device[6].

As can be seen from the case, the *Director* first analyzes the input log to give key points, and then the *Executor* generates detailed knowledge based on the guidance of these key points. Following evaluation criteria, the *Evaluator* evaluates the quality of the generated knowledge and provides feedback to the *Executor* for refinement. The evaluation feedback shows that the content generated by the *Executor* does not fulfill completeness and consistency. Specifically, the generated knowledge lacks an introduction to the procedure step, and the explanation of the severity level is not rigorous. In addition, there is also an uninformative explanation in the generated knowledge,

[5]https://openai.com/api/pricing/

[6]https://support.huawei.com/enterprise/en/doc/EDOC1100332302/fb54bda3

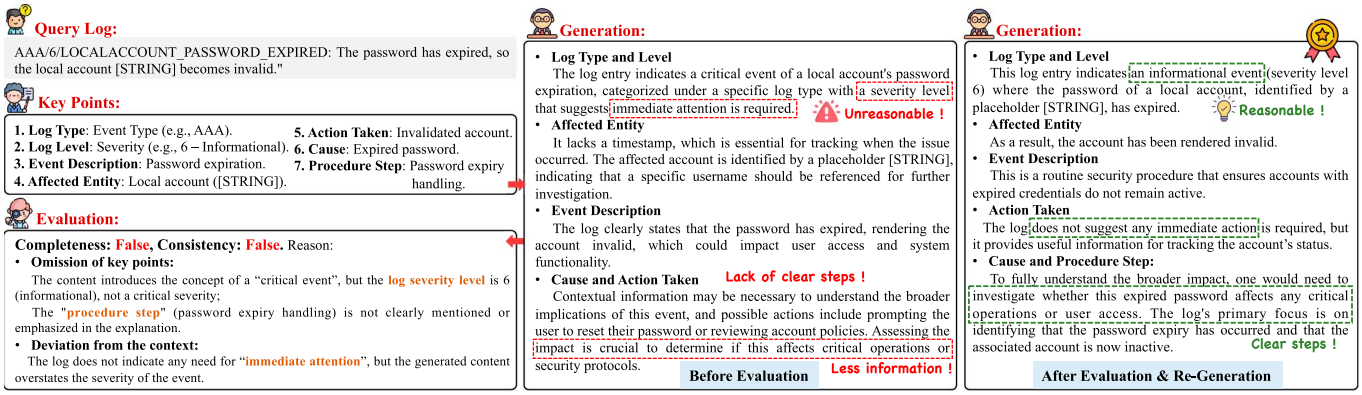
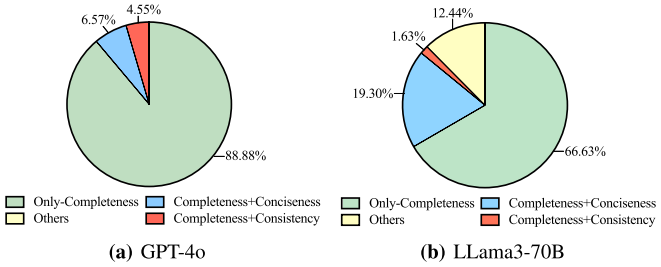


Fig. 14. An complete workflow of the MEC framework to construct knowledge.



(a) GPT-4o (b) LLama3-70B

Fig. 15. Distributions of error categories in unsatisfactory knowledge.

which makes it more difficult for the smaller model to understand the expert knowledge. Finally, the *Executor* refines the generated knowledge based on the evaluation feedback. From the regenerated knowledge, we can find that the *Executor* not only supplements the clear introduction of the procedure step but also modifies the explanation of severity level to make it more reasonable. Overall, the regenerated knowledge is more comprehensive and rigorous, which is more conducive for the smaller model to learn expert knowledge.

To more deeply reveal the issues of knowledge generated by LLMs. We count the distributions of different error categories in unsatisfactory knowledge according to *Evaluator*'s results for GPT-4o and LLama3-70B, respectively. As shown in Fig. 15, it can be found that a significant portion of the inadequacies stems from issues related to completeness. The prevalence of completeness-related errors underscores the challenge faced by the single LLM in producing comprehensive knowledge autonomously. Moreover, alongside completeness issues, a smaller yet notable proportion of unsatisfactory knowledge is attributed to deficiencies in conciseness and consistency. This finding underscores the significance of collaborative frameworks that integrate diverse roles, each contributing unique perspectives and expertise to compensate for the capacity deficiencies of the single LLM and improve the completeness and accuracy of generated knowledge.

2) Comparison of Knowledge From Different Sources: To further intuitively analyze the quality of the expert knowledge constructed with the MEC framework, we compare it with the knowledge collected from documentation and the COT-based approach. As shown in Fig. 16, we collect a Cisco log[7] as the case and give the corresponding generated knowledge based on the COT and MEC methods.

As can be seen from these cases, the knowledge provided by the documentation is too simple, which provides a generic and limited understanding of the log without delving into specific details or potential causes. Although COT-based knowledge offers a more detailed explanation of the log, compared to MEC-based knowledge, there is still the problem of inadequate knowledge and lack of insight. In contrast, the knowledge derived from the MEC framework excels in its comprehensive analysis and insights. By explicitly outlining the specifics of the log error, including the MTS queue failure and the placeholders for queue and error message, the MEC-based knowledge provides a deeper understanding of the issue at hand. Furthermore, it goes beyond mere description to identify potential root causes such as network connectivity issues. The comparison highlights the clear advantage of the MEC framework in expert knowledge generation. Its collaborative nature allows for the integration of diverse expertise and perspectives, resulting in a more comprehensive analysis of complex logs.

To quantify the differences among the various forms of knowledge, we employ the evaluation criteria of the MEC framework to evaluate the knowledge acquired by the documentation and the knowledge constructed based on COT in the network device logs. The evaluation results are shown in Fig. 17, revealing a notable contrast in quality between the different sources of knowledge. It is apparent from the analysis that a significant proportion of knowledge sourced from documentation falls short of meeting the evaluation requirements. This deficiency can be attributed to the inherent limitations of documentation-derived knowledge, which often lacks depth and detail, offering only the most straightforward description of the logs. Combined with Fig. 12(a), this suggests that the knowledge acquired from MEC based on LLMs is comparable or even better than the documentation. Considering the lack of documentation support for most logs and the high cost of

[7]https://www.cisco.com/c/en/us/td/docs/switches/datacenter/sw/routing_messages/reference/7k_rout_mess_ref_book/7k_rout_mess_ref_2mess.html

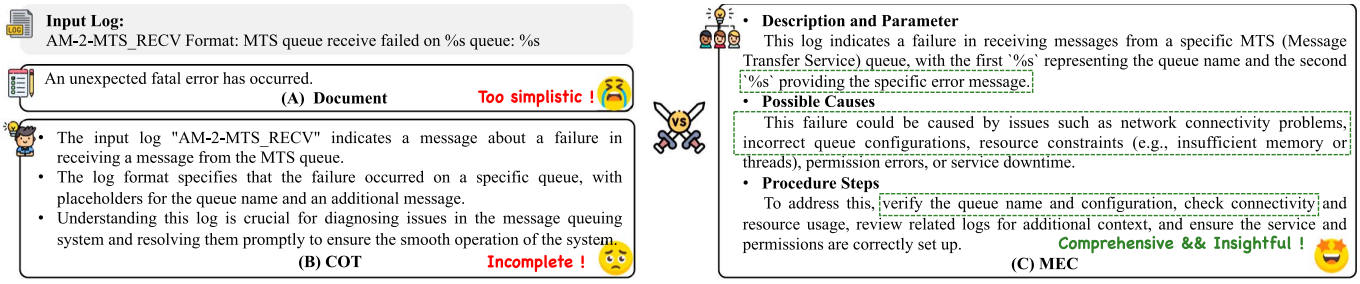


Fig. 16. Example of knowledge generated by GPT-4o and retrieved from documentation.

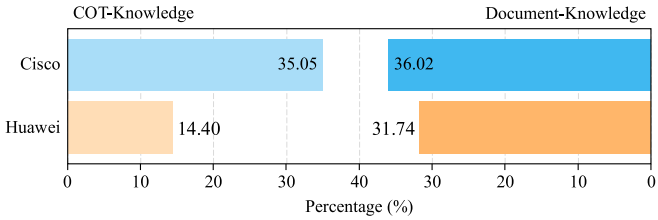


Fig. 17. Proportion of unsatisfactory knowledge, where we employ the evaluation criteria of the MEC framework to evaluate the knowledge.

manually constructing knowledge, this provides a feasible way to gain a wealth of practical knowledge automatically.

Despite the advantages of the MEC framework in knowledge generation, domain-specific documentation continues to hold an indispensable role. LLMs remain struggling with knowledge gaps, especially when dealing with customized modules and terminology, leading to misconceptions. In such cases, documentation can bridge these gaps, providing essential context and expertise to enhance the understanding of LLMs. This relationship allows LLMs to enrich documentation content and, in turn, enables documentation to improve the professionalism and accuracy of LLMs. We will optimize the integration of domain documentation and LLM to further enhance the knowledge acquisition process in the future.

D. Threats to Validity

1) Construct Validity: Since LLMs are black box models, LLMs pose a risk of generating output content that may be unreasonable, potentially impacting the accurate interpretation of logs. First, to counteract the inherent randomness in the inference process of LLMs, we set the model temperature to 0. This step ensures that LLMs consistently produce results for the same inputs, mitigating variability in output. Second, to mitigate hallucinations of LLMs, we propose a multi-expert collaboration framework, which can work better with the power of cooperation and interaction. In particular, we design an *Evaluator* in this framework, which checks the generated knowledge by evaluating completeness, consistency, and conciseness. In addition, to address the issue of ambiguous and inconsistent evaluations by LLMs under the reference-free setting, we construct contrastive examples consisting of positive and negative knowledge of the same log. By presenting these contrastive examples to the *Evaluator* as reference examples, we steer the evaluation of LLMs toward our desired objectives. In cases where issues are identified, the *Executor* within the framework revises or refines the generated content based on the feedback received. This iterative process helps improve the quality and reliability of the generated knowledge. We also verify the importance of the *Evaluator* for acquiring high-quality knowledge in ablation studies.

2) Internal Validity: It is widely agreed that the performance of DL models is significantly affected by hyperparameters. Due to the limited computation resources, we do not search for the optimal hyperparameter settings and instead follow the empirical settings. We acknowledge that further fine-tuning of these hyperparameters may yield better results.

3) External Validity: From the perspective of enterprises, utilizing external LLMs to acquire expert knowledge may cause leakage of user privacy and internal information in logs. To alleviate this issue, we suggest that enterprises can utilize log parsing as a potential approach to mitigate privacy risks while removing duplicate logs. Log parsing can be used to extract log templates, which help in removing sensitive information from logs. By using these sanitized templates, enterprises can safely leverage proprietary LLMs to acquire expert knowledge without exposing user privacy or internal information. On the other hand, LUK is a general framework that can combine any LLMs, and users can also employ their LLMs to acquire knowledge to enhance a small model for solving a specific problem. For example, apart from invoking the proprietary LLMs, we also verify the effectiveness of LUK by deploying LLama3-70B in our experiments, which is an open-source LLM.

From the perspective of evaluation, in RQ3, we simulate log instability by injecting controlled noise into original logs due to the lack of real-world evolving log datasets. However, when the noise ratio increases to 40%, especially for short log messages with fewer tokens, the semantic meaning of the original logs may be modified, deviating from the ground truth and potentially impacting evaluation validity. Moreover, our synthetic approach cannot fully capture the diversity of real-world log changes. In future work, we will collect more real-world logs to explore other possible types of changes.

VI. Conclusion

In conclusion, this paper introduces LUK, a novel knowledge enhancement framework to improve log understanding with expert knowledge from LLMs. Unlike existing LLM-based

log analysis studies that directly use the in-context learning of LLMs, LUK first acquires expert knowledge from LLMs, then enhances the log pre-training with the corresponding expert knowledge on a smaller pre-trained language model. Finally, the enhanced smaller pre-trained model for logs can be fine-tuned to solve downstream log analysis tasks. Compared to existing models, LUK achieves state-of-the-art performance on different log analysis tasks, which proves that expert knowledge from LLMs can be used more effectively to understand logs.

Data Availability Statement

Our source code and detailed experimental data are available at https://github.com/LeaperOvO/LUK.

Acknowledgment

The computations in this research were performed using the CFFF platform of Fudan University. We express our sincere appreciation to all anonymous reviewers for their constructive feedback.

References

[1] X. Zhang et al., "Onion: Identifying incident-indicating logs for cloud systems," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found.s Softw. Eng.*, 2021, pp. 1253–1263.

[2] J. Zhu et al., "Tools and benchmarks for automated log parsing," in *Proc. IEEE/ACM 41st Int. Conf. Softw. Eng., Softw. Eng. Pract. (ICSE-SEIP),* Piscataway, NJ, USA: IEEE Press, 2019, pp. 121–130.

[3] H. Dai, H. Li, C.-S. Chen, W. Shang, and T.-H. Chen, "*Logram*: Efficient log parsing using n n-gram dictionaries," *IEEE Trans. Softw. Eng.*, vol. 48, no. 3, pp. 879–892, Mar. 2022.

[4] S. Nedelkoski, J. Bogatinovski, A. Acker, J. Cardoso, and O. Kao, "Self-supervised log parsing," in *Proc. Mach. Learn. Knowl. Discovery Databases, Appl. Data Sci. Track, Eur. Conf. (ECML PKDD)*, Ghent, Belgium. New York, NY, USA: Springer-Verlag, 2021, pp. 122–138.

[5] Y. Liu et al., "UniParser: A unified log parser for heterogeneous log data," in *Proc. ACM Web Conf.*, 2022, pp. 1893–1901.

[6] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1285–1298.

[7] W. Meng et al., "LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs," in *Proc. Int. Joint Conf. Artif. Intell.*, 2019, vol. 19, no. 7, pp. 4739–4745.

[8] X. Li, P. Chen, L. Jing, Z. He, and G. Yu, "SwissLog: Robust and unified deep learning based log anomaly detection for diverse faults," in *Proc. IEEE 31st Int. Symp. Softw. Rel. Eng. (ISSRE),* Piscataway, NJ, USA: IEEE Press, 2020, pp. 92–103.

[9] S. Nedelkoski, J. Bogatinovski, A. Acker, J. Cardoso, and O. Kao, "Self-attentive classification-based anomaly detection in unstructured logs," in *Proc. IEEE Int. Conf. Data Mining (ICDM),* Piscataway, NJ, USA: IEEE Press, 2020, pp. 1196–1201.

[10] S. Huang et al., "HitAnomaly: Hierarchical transformers for anomaly detection in system log," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 4, pp. 2064–2076, Dec. 2020.

[11] X. Han and S. Yuan, "Unsupervised cross-system log anomaly detection via domain adaptation," in *Proc. 30th ACM Int. Conf. Inf. Knowl. Manage.*, 2021, pp. 3068–3072.

[12] V.-H. Le and H. Zhang, "Log-based anomaly detection with deep learning: How far are we?" in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 1356–1367.

[13] S. Lu, B. Rao, X. Wei, B. Tak, L. Wang, and L. Wang, "Log-based abnormal task detection and root cause analysis for Spark," in *Proc. IEEE Int. Conf. Web Services (ICWS),* Piscataway, NJ, USA: IEEE Press, 2017, pp. 389–396.

[14] L. Wang, N. Zhao, J. Chen, P. Li, W. Zhang, and K. Sui, "Root-cause metric location for microservice systems via log anomaly detection," in *Proc. IEEE Int. Conf. Web Services (ICWS),* Piscataway, NJ, USA: IEEE Press, 2020, pp. 142–150.

[15] J. Soldani and A. Brogi, "Anomaly detection and failure root cause analysis in (micro) service-based cloud applications: A survey," *ACM Comput. Surv. (CSUR)*, vol. 55, no. 3, pp. 1–39, 2022.

[16] S. Zhang et al., "PreFix: Switch failure prediction in datacenter networks," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 2, no. 1, pp. 1–29, 2018.

[17] J. Gao, H. Wang, and H. Shen, "Task failure prediction in cloud data centers using deep learning," *IEEE Trans. Services Comput.*, vol. 15, no. 3, pp. 1411–1422, May/Jun. 2022.

[18] D. Cotroneo, L. De Simone, P. Liguori, R. Natella, and N. Bidokhti, "How bad can a bug get? An empirical analysis of software failures in the OpenStack cloud computing platform," in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2019, pp. 200–211.

[19] J. Achiam et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.

[20] Y. Zhu et al., "UniLog: Deploy one model and specialize it for all log analysis tasks," 2021, *arXiv:2112.03159*.

[21] S. Tao et al., "Biglog: Unsupervised large-scale pre-training for a unified log representation," in *Proc. IEEE/ACM 31st Int. Symp. Qual. Service (IWQoS),* Piscataway, NJ, USA: IEEE Press, 2023, pp. 1–11.

[22] L. Ma et al., "KnowLog: Knowledge enhanced pre-trained language model for log understanding," in *Proc. 46th IEEE/ACM Int. Conf. Softw. Eng.*, 2024, pp. 1–13.

[23] V.-H. Le and H. Zhang, "Log parsing with prompt-based few-shot learning," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE),* Piscataway, NJ, USA: IEEE Press, 2023, pp. 2438–2449.

[24] Y. Lee, J. Kim, and P. Kang, "LAnoBERT: System log anomaly detection based on BERT masked language model," *Appl. Soft Comput.*, vol. 146, Oct. 2023, Art. no. 110689.

[25] C. Almodovar, F. Sabrina, S. Karimi, and S. Azad, "LogFiT: Log anomaly detection using fine-tuned language models," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 2, pp. 1715–1723, Apr. 2024.

[26] J. Xu, R. Yang, Y. Huo, C. Zhang, and P. He, "DivLog: Log parsing with prompt enhanced in-context learning," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng. (ICSE),* Washington, DC, USA: IEEE Computer Society, 2024, pp. 983–983.

[27] J. Xu et al., "UniLog: Automatic logging via LLM and in-context learning," in *Proc. 46th IEEE/ACM Int. Conf. Softw. Eng.*, 2024, pp. 1–12.

[28] Y. Liu et al., "LogPrompt: Prompt engineering towards zero-shot and interpretable log analysis," 2023, *arXiv:2308.07610*.

[29] Z. Jiang et al., "LLMParser: A LLM-based log parsing framework," 2023, *arXiv:2310.01796*.

[30] H. Guo et al., "Lemur: Log parsing with entropy sampling and chain-of-thought merging," 2024, *arXiv:2402.18205*.

[31] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Human Lang. Technol. (Volume 1 Long and Short Papers)*, 2019, pp. 4171–4186.

[32] Q. Dong et al., "A survey on in-context learning," 2022, *arXiv:2301.00234*.

[33] Y. Lu, M. Bartolo, A. Moore, S. Riedel, and P. Stenetorp, "Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity," in *Proc. 60th Annu. Meeting Assoc. Comput. Linguistics (Volume 1: Long Papers)*, 2022, pp. 8086–8098,

[34] X. Wang et al., "Spine: A scalable log parser with feedback guidance," in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2022, pp. 1198–1208.

[35] X. Li, H. Zhang, V.-H. Le, and P. Chen, "LogShrink: Effective log compression by leveraging commonality and variability of log data," in *Proc. 46th IEEE/ACM Int. Conf. Softw. Eng.*, 2024, pp. 1–12.

[36] Y. Wang, K. Chen, H. Tan, and K. Guo, "Tabi: An efficient multi-level inference system for large language models," in *Proc. 18th Eur. Conf. Comput. Syst.*, 2023, pp. 233–248.

[37] Z. Ma, A. R. Chen, D. J. Kim, T.-H. Chen, and S. Wang, "LLMParser: An exploratory study on using large language models for log parsing," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, 2024, pp. 1–13.

[38] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, "Loghub: A large collection of system log datasets for ai-driven log analytics," in *Proc. IEEE 34th Int. Symp. Softw. Rel. Eng. (ISSRE),* Piscataway, NJ, USA: IEEE Press, 2023, pp. 355–366.

[39] Y. Peng, C. Wang, W. Wang, C. Gao, and M. R. Lyu, "Generative type inference for Python," in *Proc. 38th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE),* Piscataway, NJ, USA: IEEE Press, 2023, pp. 988–999.

[40] A. Mastropaolo, L. Pascarella, and G. Bavota, "Using deep learning to generate complete log statements," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 2279–2290.

[41] Y. Li et al., "Exploring the effectiveness of LLMs in automated logging generation: An empirical study," 2023, *arXiv:2307.05950*.

[42] N. Mündler, J. He, S. Jenko, and M. Vechev, "Self-contradictory hallucinations of large language models: Evaluation, detection and mitigation," 2023. *arXiv:2305.15852*.

[43] P. Manakul, A. Liusie, and M. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2023, pp. 9004–9017.

[44] X. Xu et al., "A survey on knowledge distillation of large language models," 2024, *arXiv:2402.13116*.

[45] X. Zhu, J. Li, Y. Liu, C. Ma, and W. Wang, "Distilling mathematical reasoning capabilities into small language models," *Neural Netw.*, vol. 179, Nov. 2024, Art. no. 106594.

[46] C.-Y. Hsieh et al., "Distilling step-by-step! Outperforming larger language models with less training data and smaller model sizes," in *Proc. Findings Assoc. Comput. Linguistics (ACL)*, 2023, pp. 8003–8017.

[47] K. Shridhar, A. Stolfo, and M. Sachan, "Distilling reasoning capabilities into smaller language models," in *Proc. Findings Assoc. Comput. Linguistics (ACL)*, 2023, pp. 7059–7073.

[48] J. Wei et al., "Emergent abilities of large language models," 2022, *arXiv:2206.07682*.

[49] S. Lu, I. Bigoulaeva, R. Sachdeva, H. T. Madabushi, and I. Gurevych, "Are emergent abilities in large language models just in-context learning?" 2023, *arXiv:2309.01809*.

[50] K. Petersen, C. Wohlin, and D. Baca, "The waterfall model in large-scale development," in *Proc. Product-Focused Softw. Process Improvement, 10th Int. Conf. (PROFES)*, Oulu, Finland. New York, NY, USA: Springer-Verlag, Jun. 15–17, 2009, pp. 386–400.

[51] F. Bai, A. Ritter, and W. Xu, "Pre-train or annotate? Domain adaptation with a constrained budget," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2021, pp. 5002–5015.

[52] C. Niu, C. Li, V. Ng, J. Ge, L. Huang, and B. Luo, "SPT-code: sequence-to-sequence pre-training for learning source code representations," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 2006–2018.

[53] Y. Gu et al., "Domain-specific language model pretraining for biomedical natural language processing," *ACM Trans. Comput. Healthcare (HEALTH)*, vol. 3, no. 1, pp. 1–23, 2021.

[54] I. Beltagy, K. Lo, and A. Cohan, "SciBERT: A pretrained language model for scientific text," 2019, *arXiv:1903.10676*.

[55] S. Gururangan et al., "Don't stop pretraining: Adapt language models to domains and tasks," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics*, 2020, pp. 8342–8360.

[56] L. Yang et al., "Supervised knowledge makes large language models better in-context learners," 2023, *arXiv:2312.15918*.

[57] G. Juneja, S. Dutta, S. Chakrabarti, S. Manchanda, and T. Chakraborty, "Small language models fine-tuned to coordinate larger language models improve complex reasoning," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2023, pp. 3675–3691.

[58] H. Guo, S. Yuan, and X. Wu, "LogBERT: Log anomaly detection via BERT," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN),* Piscataway, NJ, USA: IEEE Press, 2021, pp. 1–8.

[59] E. Kasneci et al., "Chatgpt for good? On opportunities and challenges of large language models for education," *Learn. Individual Differences*, vol. 103, Apr. 2023, Art. no. 102274.

[60] J. White et al., "A prompt pattern catalog to enhance prompt engineering with ChatGPT," 2023, *arXiv:2302.11382*.

[61] S. Feng and C. Chen, "Prompting is all you need: Automated Android bug replay with large language models," in *Proc. 46th IEEE/ACM Int. Conf. Softw. Eng.*, 2024, pp. 1–13.

[62] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, "Software testing with large language models: Survey, landscape, and vision," *IEEE Trans. Softw. Eng.*, vol. 50, no. 4, pp. 911–936, Apr. 2024.

[63] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 1877–1901.

[64] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, vol. 35, pp. 24824–24837.

[65] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "OPTQ: Accurate quantization for generative pre-trained transformers," in *Proc. 11th Int. Conf. Learn. Representations*, 2022. Available: https://openreview.net/forum?id=tcbBPnfwxS

[66] B. Goertzel, "Cognitive synergy: A universal principle for feasible general intelligence," in *Proc. 8th IEEE Int. Conf. Cogn. Inform.*, Piscataway, NJ, USA: IEEE Press, 2009, pp. 464–468.

[67] J. R. Katzenbach and D. K. Smith, *The Wisdom of Teams: Creating the High-Performance Organization.* Harvard Business Review Press, 2015.

[68] J. Zhou et al., "Instruction-following evaluation for large language models," 2023, *arXiv:2311.07911*.

[69] Z. Li, B. Peng, P. He, and X. Yan, "Evaluating the instruction-following robustness of large language models to prompt injection," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2024, pp. 557–568.

[70] S. Dhuliawala et al., "Chain-of-verification reduces hallucination in large language models," 2023, *arXiv:2309.11495*.

[71] L. Huang et al., "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Trans. Inf. Syst.*, vol. 43, no. 2, pp. 42:1–42:55, 2025, doi:10.1145/3703155.

[72] L. Zheng et al., "Judging LLM-as-a-judge with MT-bench and chatbot arena," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, vol. 36, pp. 46595–46623.

[73] P. Ke et al., "CritiqueLLM: Towards an informative critique generation model for evaluation of large language model generation," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (Volume 1: Long Papers)*, 2024, pp. 13034–13054.

[74] X. Gao, Y. Zhang, M. Galley, C. Brockett, and W. B. Dolan, "Dialogue response ranking training with large-scale human feedback data," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP),* 2020, pp. 386–395.

[75] A. Radford et al., "Learning transferable visual models from natural language supervision," in *Int. Conf. Mach. Learn.*, 2021, pp. 8748–8763.

[76] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods Natural Lang. Process. 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP),* 2019, pp. 3982–3992.

[77] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proc. IEEE Int. Conf. Web Services (ICWS),* Piscataway, NJ, USA: IEEE Press, 2017, pp. 33–40.

[78] A. Dubey et al., "The Llama 3 herd of models," 2024, *arXiv:2407.21783*.

[79] S. He, P. He, Z. Chen, T. Yang, Y. Su, and M. R. Lyu, "A survey on automated log analysis for reliability engineering," *ACM Comput. Surv. (CSUR)*, vol. 54, no. 6, pp. 1–37, 2021.

[80] N. Zhao et al., "An empirical investigation of practical log anomaly detection for online service systems," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2021, pp. 1404–1415.

[81] B. Yu et al., "Deep learning or classical machine learning? An empirical study on log-based anomaly detection," in *Proc. 46th IEEE/ACM Int. Conf. Softw. Eng.*, 2024, pp. 1–13.

[82] V.-H. Le and H. Zhang, "Log-based anomaly detection without log parsing," in *Proc. 36th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE),* Piscataway, NJ, USA: IEEE Press, 2021, pp. 492–504.

[83] M. S. Sorower, "A literature survey on algorithms for multi-label learning," *Oregon State Univ., Corvallis*, vol. 18, no. 1, p. 25, 2010.

[84] E. Fix and J. L. Hodges, "Discriminatory analysis: Nonparametric discrimination, consistency properties," *Int. Stat. Rev.,*, vol. 57, no. 3, pp. 238–247, 1989.

[85] X. Zhang et al., "Robust log-based anomaly detection on unstable log data," in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2019, pp. 807–817.

[86] S. Lu, X. Wei, Y. Li, and L. Wang, "Detecting anomaly in big data system logs using convolutional neural network," in *Proc. IEEE 16th Int. Conf. Dependable, Autonomic Secure Comput., 16th Int. Conf. Pervasive Intell. Comput., 4th Int. Conf. Big Data Intell. Comput. Cyber Sci. Technol. Congr. (DASC/PiCom/DataCom/CyberSciTech*), Piscataway, NJ, USA: IEEE Press, 2018, pp. 151–158,

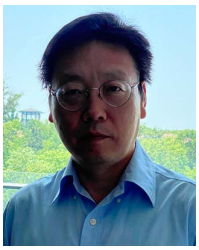
[87] R. Taori et al. "Stanford alpaca: An instruction-following Llama model." GitHub. [Online]. Available: https://Github.com/Tatsu-Lab/Stanford_Alpaca

[88] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, no. 140, pp. 1–67, 2020.

[89] Z. Zhao, E. Wallace, S. Feng, D. Klein, and S. Singh, "Calibrate before use: Improving few-shot performance of language models," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 12697–12706.

[90] S. Kabinna, C.-P. Bezemer, W. Shang, M. D. Syer, and A. E. Hassan, "Examining the stability of logging statements," *Empirical Softw. Eng.*, vol. 23, pp. 290–333, Feb. 2018.

Lipeng Ma received the B.S. degree in information security from Nanjing University of Posts and Telecommunications (NUPT), Nanjing, China, in 2019. He is currently working toward the Ph.D. degree in computer science with Fudan University. His research interests include knowledge graph application, large language models, software engineering, and AIOps.



Weidong Yang (Member, IEEE) received the Ph.D. degree in software engineering from Xidian University, in 1999. From 1999 to 2001, he was a Postdoctoral Researcher with the School of Computer Science, Fudan University. He is a Professor with the School of Computer Science, Fudan University, Shanghai, China. His research interests include big data, knowledge engineering, database and data mining, and software engineering.

Sihang Jiang received the B.Sc. degree in information security from Fudan University, in 2017, and the Ph.D. degree in software engineering from Fudan University, in 2024. He is currently a Postdoctoral Researcher with Fudan University. His research interests include knowledge graph, large language models, and AIOps.

Ben Fei (Member, IEEE) received the M.S. degree from the Department of Materials Science, Fudan University, Shanghai, China, in 2021, and the Ph.D. degree from the School of Computer Science, Fudan University, in 2024. He is currently a Postdoctoral with The Chinese University of Hong Kong. His research interests include generative models, large language models, 3-D computer vision, and AI for science. He is an IEEE Young Professional.

Mingjie Zhou is currently working toward the Ph.D. degree with Fudan University, Shanghai, China. His research interests include AIOps and knowledge graph. His work has been published in data mining and software engineering international conferences.

Shuhao Li received the master's degree in computer technology from Guangzhou University, Guangzhou, China, in 2023. He is currently working toward the Ph.D. degree with the School of Computer Science, Fudan University, Shanghai, China. His research interests include spatio-temporal data mining, anomaly detection, and large language models.

Mingyu Zhao received the Ph.D. degree in computer science from Fudan University, China, in 2024. He is currently an Assistant Professor with the Northwestern Polytechnical University, China. His research interests include data mining and machine learning.

Bo Xu is an Associate Professor with the School of Computer Science and Technology, Donghua University. His research interests include knowledge graph, large language models, and intelligent operations and maintenance.

Yanghua Xiao received the Ph.D. degree in software theory from Fudan University, Shanghai, China, in 2009. He is a Professor of computer science with Fudan University. He is the Director of Knowledge Works Lab, Fudan University. His research interests include big data management and mining, graph database, knowledge graph. He was a Visiting Professor with the Human Genome Sequencing Center, Baylor College Medicine, and a Visiting Researcher with Microsoft Research Asia.